Package 'breaktest'

October 22, 2026

Version 0.3.0

Date 2026-09-21

Title Set of unit root and cointegration tests

Author Anton Skrobotov [aut, ctb],
Alexey Tsarev [aut, ctb],
Vadim Zyamalov [aut, cre, ctb]

Maintainer Vadim Zyamalov <zyamalov@ranepa.ru>

Depends R (>= 3.5.0)

Encoding UTF-8

Imports doSNOW, foreach, stringr, Rfast

Description This package contains a set of different unit root and cointegration tests in the presence of structural breaks in the data.

License GPL (>= 2)

URL https://github.com/d9d6ka/breaktest

BugReports https://github.com/d9d6ka/breaktest/issues

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.0.0

Contents

bootstrap . . . 2
breaktest.KP . . . 3
coint.conf.sets . . . 4
coint.conf.sets.KY . . . 6
coint.CSS . . . 7
coint.GH . . . 10
coint.PR . . . 11
coint.VECM . . . 12
cs.bubble.emerge . . . 13
fcst.AveLR . . . 15
fcst.AveW . . . 16
fcst.BP.simulate . . . 17
fcst.CIavg . . . 18
fcst.compare . . . 20
fcst.conf.int.Bai . . . 22

fcst.conf.set.EoMorley . . . 23
fcst.DGP . . . 25
fcst.ExpS . . . 27
fcst.FGLS . . . 28
fcst.forecast . . . 30
fcst.select . . . 32
fcst.stein . . . 33
fcst.test.BP . . . 35
fcst.test.Wald . . . 37
fcst.WA . . . 38
fcst.WGLS.cv . . . 40
get.cv.KY . . . 41
get.pv.Wald . . . 42
info.criterions . . . 43
KP.seq.statistic . . . 44
MDF.1br . . . 45
MDF.mlt . . . 46
plot_bubble . . . 46
plot_bubble_confsets . . . 47
plot_confsets . . . 49
PY.statistic . . . 51
uroot.ADF . . . 52
uroot.CHLT . . . 53
uroot.HLT . . . 54
uroot.robust . . . 55
uroot.robust.mlt . . . 55
uroot.SADF . . . 56

Index **61**

`bootstrap` *Functions to receive bootstrapped p-values.*

Description

`bootstrap` is a generic function aimed to handle objects of classes provided in this package.

Usage

```
bootstrap(obj, ...)

## S3 method for class 'bt_adf'
bootstrap(obj, iter = 999)

## S3 method for class 'bt_kpss'
bootstrap(obj, iter = 999, boot.type = "sample")

## S3 method for class 'bt_SADF'
bootstrap(obj, iter = 999)

## S3 method for class 'bt_GSADF'
bootstrap(obj, iter = 999)
```

```
## S3 method for class 'bt_mdfCHLT'
bootstrap(obj, iter = 999, y)
```

Arguments

obj	An object of one of the following classes: • bt_adf, • bt_kpss, • bt_SADF and bt_GSADF, • bt_mdfCHLT.
iter	Number of bootstrapping iterations.
boot.type	Type of bootstrapping: • "sample"sampling from residuals with replacement, • "Cavaliere-Taylor"multiplying residuals by $N(0,1)$-distributed variable, • "Rademacher"multiplying residuals by Rademacher-distributed variable.

Details

bootstrap is a generic function aimed to handle objects of classes provided in this package. Normally one should use special function parameters to obtain bootstrapped critical values or p-values (depends on test). If later you decide to calculate one then you call bootstrap function on the object with corresponding results.

Value

Bootstrapped critical value or p-value.

---

breaktest.KP *Kejrival-Perron procedure of breaks number detection*

Description

Kejrival-Perron procedure of breaks number detection

Usage

```
breaktest.KP(y, const = FALSE, breaks = 1, criterion = "bic", trim = 0.15)
```

Arguments

y	An input series of interest.
const	Whether the break in constant is allowed.
breaks	Number of breaks.
criterion	Needed information criterion: aic, bic, hq or lwz.
trim	A trimming value for a possible break date bounds.

Value

The estimated optimal break point.

References

Kejriwal, Mohitosh, and Pierre Perron. "A Sequential Procedure to Determine the Number of Breaks in Trend with an Integrated or Stationary Noise Component: Determination of Number of Breaks in Trend." Journal of Time Series Analysis 31, no. 5 (September 2010): 305–28. https://doi.org/10.1111/j.1467-9892.2010.00666.x.

Skrobotov, Anton. "Survey on structural breaks and unit root tests." Applied Econometrics 58 (2020): 96–141. https://doi.org/10.22394/1993-7601-2020-58-96-141.

`coint.conf.sets` *Confidence sets for the break date in cointegrating regressions*

Description

Procedure to construct a confidence set for the change point in cointegrating regressions.

Usage

```
coint.conf.sets(
  y,
  trend = FALSE,
  zb = NULL,
  zf = NULL,
  nF = NULL,
  nL = NULL,
  conf.level = 0.9,
  trim = 0.05,
  criterion = "bic"
)

cset_break_coint(
  y,
  trend,
  zb = NULL,
  zf = NULL,
  nF = NULL,
  nL = NULL,
  conf.level = c(0.9, 0.95),
  trim = 0.05,
  centering = FALSE,
  criterion = "bic"
)
```

Arguments

`y`	A time series of interest.
`trend`	Whether the trend is to be included.

zb	I(1) regressors with break.
zf	I(1) regressors without break.
nF, nL	Number of leads and lags of z regressors. If any is NULL then both are estimated using informational criterion.
conf.level	Confidence level to obtain appropriate critical values.
trim	The trimming parameter to find the lower and upper bounds of possible break date.
criterion	A criterion for lead and lag number estimation.
centering	Boolean. Whether the data should be detrended during the procedure.

Details

The function constructs confidence sets for a break date in cointegrating regressions by inverting a test for the break location, which is obtained by maximizing the weighted average of power. It is found in Kurozumi and Skrobotov (2018) that the limiting distribution of the test depends on the number of I(1) regressors whose coefficients sustain structural change and the number of I(1) regressors whose coefficients are fixed throughout the sample. By Monte Carlo simulations, the authors of the original paper showed that compared with a confidence interval developed by using the existing method based on the limiting distribution of the break point estimator under the assumption of the shrinking shift, the confidence set proposed in the present paper has a more accurate coverage rate, while the length of the confidence set is comparable.

The model underlying the confidence set construction is

$$y_t = \beta_{b,c} + \beta_{b,\tau} t + z'_{b,t}\beta_{b,z} + 1(t > [\lambda_0 T]) \left(\delta_{b,c} + \delta_{b,\tau} t + z'_{b,t}\delta_{b,z}\right) + z'_{f,t}\beta_{f,z} + u_t,$$

where $z_{b,t}$ (zb) is the vector of $I(1)$ regressors whose coefficients sustain a structural break at the unknown date $T_0 = [\lambda_0 T]$, and $z_{f,t}$ (zf) is the vector of $I(1)$ regressors whose coefficients are fixed throughout the sample. The constant and, if trend = TRUE, the linear trend are allowed to break jointly with $z_{b,t}$, corresponding to Model II-c of Kurozumi and Skrobotov (2018); setting trend = FALSE reduces the specification to the corresponding Model I-c.

Since $z_{b,t}$ and $z_{f,t}$ are $I(1)$ and endogenous, the regression is augmented with leads and lags of their first differences (dynamic OLS), so that the estimated equation becomes

$$\begin{aligned} y_t = \beta_{b,c} + \beta_{b,\tau} t + z'_{b,t}\beta_{b,z} + 1(t > [\lambda_0 T]) \left(\delta_{b,c} + \delta_{b,\tau} t + z'_{b,t}\delta_{b,z}\right) + z'_{f,t}\beta_{f,z} + \\ + \sum_{j=-nL}^{nF} \pi'_{b,j} \Delta z_{b,t-j} + \sum_{j=-nL}^{nF} \pi'_{f,j} \Delta z_{f,t-j} + u_t, \end{aligned}$$

where nF and nL are the numbers of leads and lags, respectively. If either is NULL, both are selected by the information criterion.

The confidence set for the break date T_0 is obtained by inverting a test for the break location that maximizes the weighted average power over the magnitude and location of the break, at the confidence level conf.level. The lower and upper bounds of admissible break dates are determined by the trimming parameter trim.

Value

Named list of five $T_y \times 1$ numeric matrices:

td	Time index; td[i] == -1 marks the estimated break date (all other entries equal their own time index, or 0 in the padded leading/trailing region lost to leads/lags/differencing).

cset_sup	Confidence set from the sup-test (1 = point included).
cset_avg	Confidence set from the avg-test.
cset_exp	Confidence set from the exp-test.
cset_bls	Confidence set from the Bai, Lumsdaine and Stock (1998) method.

References

Kurozumi, Eiji, and Anton Skrobotov. "Confidence Sets for the Break Date in Cointegrating Regressions." Oxford Bulletin of Economics and Statistics 80, no. 3 (2018): 514–35. https://doi.org/10.1111/obes.12223.

`coint.conf.sets.KY` *Confidence sets for a break date based on optimal tests (Kurozumi & Yamamoto, 2015).*

Description

Confidence sets for a break date based on optimal tests (Kurozumi & Yamamoto, 2015).

Usage

```
coint.conf.sets.KY(y, zb, zf = NULL, trim = 0.05, c_level = c(0.9, 0.95))
```

Arguments

y	$T_y \times 1$ numeric matrix or vector, the dependent time series.
zb	$T_y \times pzb$ matrix of I(1) regressors whose coefficient sustains a structural change; NULL if there are none.
zf	$T_y \times pzf$ matrix of I(1) regressors whose coefficient is fixed (does not change at the break); NULL if there are none.
trim	Trimming share defining lower and upper bounds of possible break moments.
c_level	Numeric, 0.90 or 0.95. Confidence level.

Value

Named list of five $T_y \times 1$ numeric matrices (1 = point included in the confidence set):

cset_em	Confidence set from the Elliott and Muller (EM) test.
cset_mem	Confidence set from the modified EM test.
cset_sup	Confidence set from the sup-test.
cset_avg	Confidence set from the avg-test.
cset_exp	Confidence set from the exp-test.

`coint.CSS` *KPSS-based stationarity and cointegration test with multiple structural breaks*

Description

Test for stationarity and cointegration in the presence of multiple structural breaks using the KPSS test.

This procedure extends the single- and two-break tests of Carrion-i-Silvestre and Sansó (2006, 2007) to the case of an arbitrary (known) number of breaks.

Usage

```
coint.CSS(
  y,
  x = NULL,
  const = TRUE,
  trend = FALSE,
  break.type,
  break.point,
  break.coint = FALSE,
  weakly.exog = TRUE,
  max.lags,
  max.leads,
  criterion = "bic",
  lr.kernel = "Kurozumi",
  lr.lag = NULL,
  boot.p = FALSE,
  boot.type = "sample",
  boot.iter = 999
)
```

Arguments

`y`	A time series of interest.
`x`	A matrix of explanatory stochastic regressors. If it's NULL then the result of this function is just a KPSS test for y.
`const, trend`	Whether a constant or trend should be included.
`break.type`	A single value or vector of • `c`: for the break in const, • `t`: for the break in trend, • `ct`: for the break in const and trend.
`break.point`	Array of structural break moments.
`break.coint`	Whether breaks in cointegrating relation are needed to be included. See Carrion-i-Silvestre & Sansó (2006) for details.
`weakly.exog`	Boolean where we specify whether the stochastic regressors are exogenous or not • `TRUE`: if the regressors are weakly exogenous,

- FALSE: if the regressors are not weakly exogenous (DOLS is used in this case).

max.lags, max.leads
: Scalars defininig the initial number of lags and leads for DOLS.

criterion
: Information criterion for DOLS lags and leads selection: aic, bic, hq, or lwz,

lr.kernel
: Kernel for calculating long-run variance

- Kurozumi for the Kurozumi's proposal, using Quadratic kernel (default).
- Bartlett: for Bartlett kernel,
- Quadratic: for Quadratic Spectral kernel,

lr.lag
: scalar showing the bandwidth of long run variance estimator. If negative or NULL then the bandwidth is selected as in Andrews (1991).

boot.p
: Whether bootstrapped p-values should be returned.

boot.type
: Method of building synthetic y:

- sample — permutations bootstrap,
- Cavaliere-Taylor — wild bootstrap with $N(0,1)$ multipliers,
- Rademacher — wild bootstrap with ± 1 multipliers with equal probabilities.

boot.iter
: The number of bootstrap iterations,

...
: A dummy parameter for technical purposes. Just ignore it.

If you want to get results like in Carrion-i-Silvestre & Sansó (2006) then you should

- provide a break.point of length 1;
- for the specification of An set const=TRUE, trend=FALSE, break.type=c("c");
- for the specification of A set const=TRUE, trend=TRUE, break.type=c("c");
- for the specification of B set const=TRUE, trend=TRUE, break.type=c("t");
- for the specification of C set const=TRUE, trend=TRUE, break.type=c("ct");
- for the specification of D set const=TRUE, trend=FALSE, break.type=c("c");
- for the specification of E set const=TRUE, trend=TRUE, break.type=c("ct"), and break.coint=TRUE.

If you want to get results like in Carrion-i-Silvestre & Sansó (2007) then you should

- provide a break.point of length 2;
- for the specification of AAn set const=TRUE, trend=FALSE, break.type=c("c", "c");
- for the specification of AA set const=TRUE, trend=TRUE, break.type=c("c", "c");
- for the specification of BB set const=TRUE, trend=TRUE, break.type=c("t", "t");
- for the specification of CC set const=TRUE, trend=TRUE, break.type=c("ct", "ct");
- for the specification of AB-BA set const=TRUE, trend=FALSE, break.type=c("c", "t");
- for the specification of AC-CA set const=TRUE, trend=TRUE, break.type=c("c", "ct");
- for the specification of BC-CB set const=TRUE, trend=TRUE, break.type=c("t", "ct").

Details

This function is a generalization of two methods proposed in Carrion-i-Silvestre and Sansó (2006, 2007).

Carrion-i-Silvestre and Sansó (2006) proposed an LM-Type statistic to test the null hypothesis of cointegration allowing for the possibility of a structural break, in both the deterministic component and the cointegration vector. The test has been designed to be used as a complement to the usual non-cointegration tests in order to obtain stronger evidence of cointegration. The cases of known and unknown break dates are considered.

Carrion-i-Silvestre and Sansó (2007) generalized the KPSS-type test for stationarity to allow for two structural breaks. Seven models have been defined depending on the way in which the structural breaks affect the time series behaviour.

At the first step, the residuals are obtained from a regression of the series y on the set of deterministic components and (if any) exogenous regressors x. Deterministic components may include

- a constant,
- a trend,
- constant with breaks $DU_{t,\tau} = 1[t > \tau]$,
- trends with breaks $DT_{t,\tau} = 1[t > \tau](t - \tau)$.

If x is weakly exogenous, the regression is estimated by OLS; otherwise, DOLS regression is applied. The procedure also allows for the break in the cointegrating equation; in that case, products of $DU_{t,\tau}$ and x are added to the regressors list.

After the residuals are obtained, the long-run variance of the KPSS test statistic is calculated using a kernel specified via `lr.kernel`. By default (`lr.kernel = "Kurozumi"`), the QS kernel is used with bandwidth selected as in Andrews (1991) and trancated following Kurozumi's (2002) proposal.

The p-value (if needed) is calculated by a bootstrapping procedure following Cavaliere and Taylor (2006).

Value

An object of class `bt_kpss` containing

- `statistic`: the value of test statistic,
- `coefficients`: OLS/DOLS estimates of the coefficients,
- `se.coefs`: standard errors of the coefficients above,
- `t.stats`: t-statistics for the coefficients above,
- `residuals`: Residuals of the model,
- `t.beta`: t-statistics for `beta`,
- `fitted.values`: fitted values of the estimated model,
- `endog`: final y vector used in the estimated model,
- `exog`: final x matrix used in the estimated model,
- `DOLS.lags`: The estimated number of lags and leads in DOLS,
- `break.type`, `break.point`, `break.coint`: breaks specification,
- `criterions`: values of the information criterions for the estimated model,
- `lags`, `leads`: number of lags and leads in the estimated model.

References

Carrion-i-Silvestre, Josep Lluís, and Andreu Sansó. "Testing the Null of Cointegration with Structural Breaks." Oxford Bulletin of Economics and Statistics 68, no. 5 (October 2006): 623–46. https://doi.org/10.1111/j.1468-0084.2006.00180.x.

Carrion-i-Silvestre, Josep Lluís, and Andreu Sansó. "The KPSS Test with Two Structural Breaks." Spanish Economic Review 9, no. 2 (May 16, 2007): 105–27. https://doi.org/10.1007/s10108-006-9017-8.

Cavaliere, G., & Robert Taylor, A. M. (2006). "Testing the Null of Co-integration in the Presence of Variance Breaks." Journal of Time Series Analysis, 27(4), 613-636.

Andrews, Donald W. K. "Heteroskedasticity and Autocorrelation Consistent Covariance Matrix Estimation." Econometrica 59, no. 3 (1991): 817–58. https://doi.org/10.2307/2938229.

Kurozumi, Eiji. "Testing for Stationarity with a Break." Journal of Econometrics 108, no. 1 (May 1, 2002): 63–99. https://doi.org/10.1016/S0304-4076(01)00106-3.

`coint.GH` *Gregory-Hansen test for the absense of cointegration*

Description

Gregory and Hansen (1996) test for the null hypothesis of no cointegration under a possible structural break at the unknown moment of time.

Usage

```
coint.GH(
  ...,
  shift = "level",
  trim = 0.15,
  max.lag = 10,
  criterion = "aic",
  add.cvals = TRUE
)
```

Arguments

`...`	Variables of interest.
`shift`	Expected break type.
`trim`	The trimming parameter to calculate break moment bounds.
`max.lag`	The maximum number of lags for the internal ADF testing.
`criterion`	The criterion for lag selection.
`add.cvals`	Whether critical values are to be returned. This argument is needed to suppress the calculation of critical values during the precalculation of tables needed for the p-values estimating.

Details

Gregory and Hansen (1996) test for the null hypothesis of no cointegration under a possible structural break at the unknown moment of time.

The authors proposed ADF- and Z-type tests, slightly modified to allow the presence of a possible regime shift. Three type of shifts are allowed:

- a shift in the constant,
- a shift in the constand with the trend included,
- and a shift in the constant and the cointegrating vector.

Critical values are calculated via the adopted MacKinnon procedure of estimating the model for the response surface.

Value

An object of type `cointGH`. It's a list of

`shift`	shift type,
`Za`	MZ_α statistic and c.v.,
`Zt`	MZ_t statistic and c.v.,
`ADF`	ADF statistic and c.v..

References

MacKinnon, James G. "Critical Values for Cointegration Tests." Working Paper. Working Paper. Economics Department, Queen's University, January 2010. https://ideas.repec.org/p/qed/wpaper/1227.html.

Gregory, Allan W., and Bruce E. Hansen. "Residual-Based Tests for Cointegration in Models with Regime Shifts." Journal of Econometrics 70, no. 1 (January 1, 1996): 99–126. https://doi.org/10.1016/0304-4076(69)41685-7.

`coint.PR` *A set of residual based tests for cointegration*

Description

A set of residual based tests for cointegration

Usage

```
coint.PR(y, x, deter, kmax = round(4 * (N/100)^(1/4)), signif = 0.05)
```

Arguments

`y`, `x`	Variables of interest. `x` can be a matrix of several variables.
`deter`	A value equal to • 1: quasi-demeaned y and x, • 2: quasi-detrended y and x, • 3: quasi-demeaned y and quasi-detrended x.

Details

Perron and Rodríguez (2016) provide generalized least-squares (GLS) detrended versions of single-equation static regression or residuals-based tests for testing whether or not non-stationary time series are cointegrated. Their approach is to consider nearly optimal tests for unit roots and to apply them in the cointegration context. Authors derive the local asymptotic power functions of all tests considered for a triangular data-generating process, imposing a directional restriction such that the regressors are pure integrated processes. Their GLS versions of the tests do indeed provide substantial power improvements over their ordinary least-squares counterparts.

The code provided is the original GAUSS code by Perron and Rodríguez ported to R.

Value

A list of:

- 7x1-matrix of test statistics values,
- estimated number of lags.

References

Perron, Pierre, and Gabriel Rodríguez. "Residuals-based Tests for Cointegration with Generalized Least-squares Detrended Data." The Econometrics Journal 19, no. 1 (February 1, 2016): 84–111. https://doi.org/10.1111/ectj.12056.

`coint.VECM` *Test of the cointegration rank with a possible break in trend*

Description

This procedure is aimed on the problem of testing for the cointegration rank of a vector autoregressive process in the case where a trend break may potentially be present in the data.

Usage

```
coint.VECM(y, r, max.lag = trunc(12 * (length(y)/100)^(1/4)), trim = 0.15)
```

Arguments

`y`	A matrix of n VAR variables.
`r`	The cointegration rank tested against the alternative of n.
`max.lag`	The maximum number of lags.
`trim`	The trimming parameter to find the lower and upper bounds of possible break dates.

Details

This procedure is aimed on the problem of testing for the cointegration rank of a vector autoregressive process in the case where a trend break may potentially be present in the data.

The test is based on estimating the quasi log likelihood for two situations, with break, and without it. The one with the smallest value is considered to be the result.

The code provided is the original GAUSS code by Harris et al. ported to R.

Value

A list of:

- the indicator of the rejection of null.
- the estimated break point.
- the estimated lag number.

References

Harris, David, Stephen J. Leybourne, and A. M. Robert Taylor. "Tests of the Co-Integration Rank in VAR Models in the Presence of a Possible Break in Trend at an Unknown Point." Journal of Econometrics, Innovations in Multiple Time Series Analysis, 192, no. 2 (June 1, 2016): 451–67. https://doi.org/10.1016/j.jeconom.2016.02.010.

`cs.bubble.emerge` *Confidence sets for the emergence, collapse, and recovery date of a bubble*

Description

Confidence sets for the emergence, collapse, and recovery date of a bubble

Usage

```
cs.bubble.emerge(y, phi_a, s2, trim = 0.1)

cs.bubble.collapse(y, lambda_e, phi_a, phi_b, s2, trim = 0.1)

cs.bubble.recovery(y, lambda_e, lambda_c, phi_a, phi_b, s2, trim = 0.1)

segments.AR1(y, trim = 0.1)

segments.NBCN(y, trim = 0.1)
```

Arguments

`y`	Time series of interest. Should be a column matrix; if not is converted internally.
`phi_a`	AR coefficient before collapse (explosive phase).
`s2`	Residual variance.
`trim`	Trimming parameter to exclude observations from breakpoint candidates.
`lambda_e`	Scalar, fraction for critical value scaling.
`phi_b`	AR coefficient after collapse (recovery phase).
`lambda_c`	Scalar lambda_c = Tc / T_yall.

Details

segments.AR1 function fits

$$y_t = \phi_1 y_{t-1} I(t \leq bp) + \phi_2 y_{t-1} I(t > bp) + e_t$$

over all candidate breakpoints in $[N \times trim, N \times (1 - trim)]$.

segments.NBCN function uses a sequential three-step search:

1. Find Tc on the full series.
2. Find Te on y[1:(Tc+1)].
3. Find Tr on y[(Tc+2):end]. Then refits the 4-regime model to get `phi_a`, `phi_b`, and `s.sq`.

Both the "estimated" and "true" parameter variants are handled via the `phi_a`, `phi_b`, and `s2` arguments, which are to be taken from the results of segments.AR1 or segments.NBCN functions.

Value

Named list of binary vectors, where 1 means that this observation is in the corresponding confidence set.

segments.AR1 returns a named list of:

- break.point: index of estimates breakpoint,
- coefficients: AR(1) coefficients for pre-bubble and exploding regimes,
- SSR: minimum value of SSR,
- s.sq: estimated variance of internal model resuduals $\hat{e}_t$.

segments.NBCN returns a named list of:

- Te_est, Tc_est, Tr_est: estimates moments of bubble emerging, break, and post-break restoration,
- phi_a, phi_b: AR(1) coefficients for exploding and post-break regimes,
- s.sq: estimated variance of internal model resuduals $\hat{e}_t$.

References

Kurozumi, Eiji, Anton Skrobotov, and Alexey Tsarev. 2022. "Time-Transformed Test for Bubbles under Non-Stationary Volatility". Journal of Financial Econometrics, advance online publication, 23. https://doi.org/10.1093/jjfinec/nbac004.

Kurozumi, Eiji, and Anton Skrobotov. 2026. "Confidence Sets for the Emergence, Collapse, and Recovery Dates of a Bubble". arXiv:2511.16172. Preprint, arXiv. https://doi.org/10.48550/arXiv.2511.16172.

`fcst.AveLR` *Forecast averaging over a quasi-likelihood ratio region*

Description

The AveLR scheme of Koo and Seo (2015). Instead of conditioning on a point estimate of the break date, the forecast is averaged over every date that is not rejected by the rescaled quasi-likelihood ratio statistic:

$$\Phi = \left\{ \gamma : \log S_N(\gamma) - \log S_N(\hat{\gamma}) \leq c\, N^{-2/3} \right\}.$$

The motivation is that when the single break model is only an approximation to a continuously changing data generating process ("strong mis-specification"), the oracle property fails and the convergence rate of the break date estimator drops to $N^{-1/3}$ or slower. A point estimate therefore cannot be trusted and the rescaling by $N^{-2/3}$ rather than N^{-1} reflects the slower rate.

Usage

```
fcst.AveLR(
  y,
  x = NULL,
  x.new = NULL,
  trim = 0.15,
  c = 5,
  break.vars = NULL,
  time.index = NULL
)
```

Arguments

`y`	A dependent variable.
`x`	Explanatory variables. If `NULL` a single constant is used.
`x.new`	A (1 x k) vector of regressor values for the forecast origin.
`trim`	A trimming fraction bounding the admissible break dates.
`c`	A tuning constant. Koo and Seo (2015) suggest 5 as a rule of thumb after extensive experimentation and report that the results are not very sensitive to it.
`break.vars`	`NULL` or all columns of `x`. The published AveLR uses unrestricted coefficients in both regimes; partial changes are rejected.
`time.index`	Original integer-period positions. Internal gaps in the estimation block are not supported.

Value

An object of class `bt_fcstCombine` with the averaged `forecast`, the LR averaging region `set` and the individual `forecasts`. The tuning constant does not define a confidence set with a nominal coverage level.

References

Koo, Bonsoo, and Myung Hwan Seo. "Structural-Break Models under Mis-Specification: Implications for Forecasting." Journal of Econometrics 188, no. 1 (2015): 166–81.

`fcst.AveW` *Forecast combination across estimation windows*

Description

Averages the forecasts produced by the same model over a set of nested estimation windows,

$$\hat{y}_{N+1} = \frac{1}{m}\sum_{i=1}^{m}\hat{y}_{N+1}(w_i), \qquad w_i = w_{min} + \frac{i-1}{m-1}(1 - w_{min}),$$

where w_i is the window length as a fraction of the sample.

Pesaran and Pick (2011) establish bias and MSFE comparisons under their break-model assumptions and study their robustness in simulations. These are not uniform guarantees for arbitrary regressions or break sizes. Averaging requires no estimate of the break date or magnitude, but the chosen range of windows still affects the forecast.

Usage

```
fcst.AveW(
  y,
  x = NULL,
  x.new = NULL,
  w.min = 0.1,
  w.max = 1,
  m = NULL,
  weights = NULL
)
```

Arguments

`y`	A dependent variable.
`x`	Explanatory variables. If `NULL` a single constant is used.
`x.new`	A (1 x k) vector of regressor values for the forecast origin.
`w.min`	The shortest window as a fraction of the sample.
`w.max`	The longest window as a fraction of the sample.
`m`	A number of windows. If `NULL` every admissible window is used, i.e. the windows are one observation apart.
`weights`	Optional finite nonnegative relative combination weights, with at least one positive entry. They are normalised to sum to one. Defaults to equal weights as in the source paper.

Value

An object of class `bt_fcstCombine`, a list of the combined `forecast`, the vector of individual `forecasts`, the window lengths used and the implied per-observation averaging weights. For an intercept-only model these reproduce the forecast directly; for a general regression they describe window membership and are not an equivalent WLS fit.

References

Pesaran, M. Hashem, and Andreas Pick. "Forecast Combination Across Estimation Windows." Journal of Business and Economic Statistics 29, no. 2 (2011): 307–18.

`fcst.BP.simulate` *Simulate the asymptotic distribution of the Boot-Pick test*

Description

Regenerates the equal performance boundary $\zeta^{1/2}(\tau_b)$ and the critical values of the `W` and `S` statistics for an arbitrary trimming and grid. This is a port of `loopbatch.m`, `invar_calc.m`, `det_equalperf.m` and `loopbatch_onlyboundary.m` and `calc_cvsimple.m` of the original MATLAB package, condensed into a single routine. A first simulation estimates the risk boundary; a second independent simulation evaluates the process at that exact interpolated boundary, without rounding the break magnitude.

The published tables shipped with the package (see fcst.BP.tables) were produced with `n.rep = 50000`, `n.grid = 1000`, `trim = 0.15` and `tau.step = 0.05`. Reproducing them takes hours; the defaults here are much lighter and are meant for experimentation.

Usage

```
fcst.BP.simulate(
  trim = 0.15,
  tau.step = 0.05,
  theta.max = 40,
  theta.step = 1,
  n.grid = 1000,
  n.rep = 2000,
  levels = c(0.01, 0.05, 0.1),
  seed = NULL,
  verbose = TRUE,
  n.cores = 1L
)
```

Arguments

`trim`	A trimming fraction bounding the admissible break dates.
`tau.step`	A step of the break fraction grid.
`theta.max, theta.step`	The range and the step of the standardised break magnitude grid.
`n.grid`	A number of points the unit interval is divided into.
`n.rep`	A number of simulated Brownian motions.
`levels`	Nominal sizes the critical values are needed for.
`seed`	An optional random seed. As in the original simulator, set.seed is applied in the calling process and simulation advances its RNG stream. Innovations for both passes are drawn there in replication order for every n.cores. Each pass uses n.grid*n.rep normal draws; the second pass shares innovations across all break fractions and both forecast targets.

verbose	Whether to report the progress.
n.cores	Number of local SOCK workers. The default 1 is sequential. Work is divided into fixed blocks of 64 replications; at most one worker per block is started. Critical values and risks agree across worker counts.

Details

Under the local-to-zero asymptotics the statistic converges to

$$Q^*(\tau) = \left[Z(\tau) + \mu(\tau;\theta_{\tau_b})\right]^2, \quad Z(\tau) = \frac{B(\tau) - \tau B(1)}{\sqrt{\tau(1-\tau)}},$$

$$\mu(\tau;\theta) = \theta\left[\sqrt{\tfrac{1-\tau}{\tau}}\tau_b I(\tau_b < \tau) + \sqrt{\tfrac{\tau}{1-\tau}}(1-\tau_b)I(\tau_b \geq \tau)\right].$$

The standardised risks of the three forecasts are then $r^{full} = B(1)+\tau_b\theta$, $r^{post} = (1-\hat{\tau})^{-1}[B(1) - B(\hat{\tau}) + \theta(\tau_b - \hat{\tau})I(\hat{\tau} < \tau_b)]$ and $r^{comb} = (1+W)^{-1}r^{full} + W(1+W)^{-1}r^{post}$.

Value

A list of:

- `tau`: the break fraction grid,
- `theta`: the break magnitude grid,
- `postbreak`, `combined`: each contains `tau`, the equal-performance boundary `zeta.sqrt` and matrices `W` / `S` with critical values by `levels`,
- `risk`: a list of the three risk profiles over the `(tau, theta)` grid,
- `n.rep`: the number of replications per pass,
- `metadata`: settings needed to reproduce and validate the calibration.

`fcst.CIavg` *Forecast averaging over a confidence set for the break date*

Description

The confidence set forecast of Altansukh and Osborn (2022):

$$\hat{y}_{N+1} = |C|^{-1}\sum_{b\in C}\hat{\beta}'_{b+1:N}x_{N+1}.$$

Every accepted break date is used once; gaps in a disconnected set are preserved. A date `b` is the last pre-break observation, so its estimation window starts at `b + 1`.

The scheme is automatically adaptive. A large break gives a narrow interval and the forecast is close to the post-break one; a small break gives a wide interval and the forecast approaches plain window averaging.

Usage

```
fcst.CIavg(
  y,
  x = NULL,
  x.new = NULL,
  interval = NULL,
  level = 0.95,
  trim = 0.1,
  conf.method = c("eo.morley", "bai"),
  set = NULL,
  time.index = NULL,
  ...
)
```

Arguments

y	A dependent variable.
x	Explanatory variables. If NULL a single constant is used.
x.new	A (1 x k) vector of regressor values for the forecast origin.
interval	An optional two-element vector with break-date interval bounds. Retained for compatibility; do not also supply set.
level	A confidence level for the set or interval.
trim	A trimming fraction bounding the admissible window starts.
conf.method	eo.morley (default) uses fcst.conf.set.EoMorley; bai retains the previous fcst.conf.int.Bai method. Ignored for supplied interval or set.
set	An optional vector of accepted break dates, possibly with gaps. Dates index the complete-case sample. Following the source rule for window starts, require b + 1 <= N * (1 - trim) and more than k post-break rows. An empty remaining set is an error. The temporal dispatcher may attach a named multiplicity attribute when distinct original dates map to the same complete-case split. Its positive integer counts preserve equal weights over the original dates.
time.index	Original increasing integer time positions, filtered together with incomplete rows before confidence-set inference. Inferred sets require a contiguous estimation block. A supplied set only defines a combination: its confidence interpretation belongs to its source.
...	Arguments for the selected confidence-set routine, e.g. variance, vcov.type and bandwidth for Eo-Morley or het for Bai.

Value

An object of class bt_fcstCombine. break.set contains the dates used, set the corresponding window starts (as in earlier versions), interval their envelope and confidence.set the inference result when computed internally. comb.weights weights the distinct split forecasts; these are equal unless original-date multiplicities were supplied by the temporal dispatcher. set.multiplicity records those counts. source.break.set retains the original dates; excluded.break.set records the dates omitted by the deterministic window rule. The filtered collection is not asserted to be a new confidence set with the original coverage.

References

Altansukh, Gantungalag, and Denise R. Osborn. "Using Structural Break Inference for Forecasting Time Series." Empirical Economics 63 (2022): 1–41.

`fcst.compare` *Compare forecasts on common origins and targets*

Description

An expanding-origin exercise using only the history available at each origin. Every method's RMSFE uses the same valid rows within a horizon. Failures are reported. If any planned row is excluded, scores are labelled conditional and nominal DM inference is suppressed.

Usage

```
fcst.compare(
  y,
  x = NULL,
  methods = c("full", "postbreak", "ppp.robust", "avew"),
  n.init = NULL,
  benchmark = "full",
  verbose = TRUE,
  ...,
  h = 1,
  type = c("direct", "iterated"),
  lags = NULL,
  constant = TRUE,
  x.future = NULL,
  index = NULL,
  method.options = list(),
  common.horizons = FALSE,
  dm.bandwidth = NULL,
  dm.hln = FALSE,
  n.cores = 1L,
  seed = NULL,
  parallel.export = list(),
  parallel.packages = character(),
  selection = NULL
)
```

Arguments

`y, x`	Data as in fcst.forecast. No rows are removed before choosing origins, constructing lags or locating targets.
`methods`	Strategies in fcst.forecast. A named vector permits aliases, e.g. c(AR.direct = "full", AR.iterated = "full").
`n.init`	Original row of the first forecast origin. The default is max(round(N/2), 10*k), using the largest design among the methods. With selection enabled a method contributes k=1 to this heuristic, avoiding an impossible starting point in high dimensions; set n.init explicitly when the conditioning set or chosen model needs a longer training history.

`benchmark` Method name or alias used for relative RMSFE and DM. Added to methods if absent.

`verbose` Report progress and the final number of failed fits.

`...` Shared method arguments. Each receives only supported options. Unknown options are errors. Prefer method.options for different settings.

`h`, `type`, `lags`, `constant`, `x.future`, `index`
See fcst.forecast. Iterated forecasts with external x require an x.future function; its inputs are always restricted to the relevant origin's history.

`method.options` Named list of method-specific argument lists, keyed by method name or alias. Overrides shared settings. Allows different types, lags, window.method, grids, trimming, supplied break dates and other options. Supplied break dates index the original response rows at every origin.

`common.horizons`
Also use the intersection of valid origins across horizons. Otherwise each horizon ends at N-h and has its own common mask.

`dm.bandwidth` Bartlett bandwidth for ordinary DM, at least h. The default is max(h, floor(P^(1/4))). With dm.hln=TRUE this must be NULL or h, selecting the rectangular estimator through h-1.

`dm.hln` Use the Harvey-Leybourne-Newbold correction, requiring an (h-1)-dependent loss differential; see fcst.test.DM.

`n.cores` Number of local SOCK workers. The default 1 is sequential. Parallel tasks are (horizon, origin) pairs; methods and CV within each task remain sequential.

`seed` Optional integer for stochastic x.future functions. Assigns one L'Ecuyer-CMRG stream per task, independently of n.cores, and restores the caller's RNG state. With NULL the sequential RNG behaviour is unchanged; random callbacks are not reproducible across parallel runs.

`parallel.export`
Named list of global objects needed by user callbacks on workers. Self-contained closures need no explicit export.

`parallel.packages`
Additional installed packages to load on workers.

`selection` Predictor-selection options as in fcst.forecast. Each method can override these in method.options. selections retains each origin/method's complete selection diagnostics.

Details

A supplied aligned design asserts that `x[t+1]` is known at t; this cannot be inferred from numeric values. Prefer explicit lags for time-series work. Forecasts are fitted even when the realised target is missing, but that target is excluded from every method's descriptive score. Any such exclusion, including a forecast failure, prevents a nominal DM statistic from being reported: conditioning on successful outcomes can alter its null distribution even if the retained dates happen to be consecutive. On a complete sample, DM still requires its loss-process central limit theorem and variance assumptions. A generic expanding-origin comparison does not establish them for every strategy or nesting relationship.

Value

For one horizon, a bt_fcstCompare containing forecasts, errors, RMSFE, rel.RMSFE, DM, origins, targets, common (the scoring mask), n.common, n.available per method, failures (origin/target/method/reason),

settings, score.scope, DM.messages, and original origin.index/target.index labels. No common valid rows gives NA scores. With multiple horizons, a bt_fcstCompareMulti contains by.horizon with these results and a long summary table.

`fcst.conf.int.Bai` *Confidence interval for the break date of Bai (1997)*

Description

Confidence interval for the break date of Bai (1997)

Usage

```
fcst.conf.int.Bai(
  y,
  x = NULL,
  bp = NULL,
  level = 0.95,
  het = TRUE,
  trim = 0.15,
  vcov.type = c("iid", "HC", "HAC"),
  bandwidth = NULL,
  kernel = "Bartlett",
  time.index = NULL
)
```

Arguments

`y`	A dependent variable.
`x`	Explanatory variables.
`bp`	An LS estimate of the break date, or an estimate with established asymptotic equivalence. If `NULL`, use fcst.window.LS. An arbitrary known date does not have this estimator's sampling distribution. The published Altansukh–Osborn forecasting algorithm may supply its own estimated date while constructing this interval on the original data.
`level`	A confidence level.
`het`	Whether the pre- and post-break second moments and variances are allowed to differ. With `FALSE` common moments are estimated by pooling regime-residual contributions and the symmetric distribution is imposed.
`trim`	Trimming used to estimate an unknown break date. Interval bounds themselves are clipped only to `[1, N-1]`.
`vcov.type`	"iid" assumes homoskedastic uncorrelated errors within each regime and uses sigma_j^2 Q_j. "HC" estimates score second moments; "HAC" estimates long-run score covariances for serially correlated errors.
`bandwidth`, `kernel`, `time.index`	Settings of HAC.variance. Original integer time positions must form a contiguous estimation block.

Details

The interval is based on

$$\frac{(\Delta' Q \Delta)^2}{\Delta' \Omega \Delta}(\hat{T}_b - T_b^0) \Rightarrow \arg\max_s \{W(s) - |s|/2\},$$

with $\Delta = \hat{\beta}_2 - \hat{\beta}_1$. For heterogeneous regimes, xi=(Delta'Q2 Delta)/(Delta'Q1 Delta) and phi=(Delta'Omega2 Delta)/(Delta'Omega1 Delta). HC and HAC use the projected scores (x_t'Delta)*u_t, rather than substituting an error-only long-run variance. These estimators require the corresponding moment, weak-dependence and shrinking-break assumptions of Bai (1997). A zero estimated break or non-positive projected score variance is unidentified for this approximation and produces an explicit error. This implementation uses stabilising moments within regimes. It does not implement Bai's separate local normalisation for deterministic trends or provide coverage after arbitrary same-sample regressor selection. A supplied date's provenance and these model assumptions cannot be verified from its integer value; passing bp asserts the stated estimator contract.

Note that the coverage of this interval is known to fall well below the nominal level when the break is small, which is exactly why Altansukh and Osborn (2022) prefer the likelihood-ratio confidence set of Eo and Morley (2015) for forecast averaging; see fcst.conf.set.EoMorley.

Value

A list of the break date bp, the interval bounds `lower`, `upper` and the scaling factor `L`.

`fcst.conf.set.EoMorley`

Eo-Morley confidence set for the date of a structural break

Description

Inverts the likelihood ratio tests in Eo and Morley (2015), equations (4)-(6). This implementation considers a univariate regression with one break in all coefficients, with either a constant or a regime-specific error variance. The accepted dates need not form a contiguous interval.

Usage

```
fcst.conf.set.EoMorley(
  y,
  x = NULL,
  level = 0.95,
  trim = 0.15,
  variance = c("regime", "constant"),
  vcov.type = c("iid", "HC", "HAC"),
  bandwidth = NULL,
  kernel = c("Bartlett", "Parzen", "Tukey-Hanning"),
  time.index = NULL
)
```

Arguments

`y`	A dependent variable. Incomplete rows of `y` and `x` are removed jointly; all returned dates refer to the complete-case sample.
`x`	Explanatory variables. If `NULL` a single constant is used.
`level`	A confidence level in (0, 1).
`trim`	A trimming fraction in (0, 0.5). Each regime must contain at least `max(ceiling(trim * N), k + 1)` observations and have full column rank.
`variance`	`regime` allows the variance to change at the coefficient break; `constant` imposes a common variance at every candidate date.
`vcov.type`	How to estimate the limit-distribution scales: • `iid`: errors independent of regressors and serially independent within each regime; uses the model-based score covariance and empirical fourth moments. With constant variance this gives Corollary 1, `omega = c(1, 1)`; • `HC`: empirical score covariances, allowing conditional heteroskedasticity; • `HAC`: within-regime long-run score covariances, allowing serial correlation.
`bandwidth`	An optional positive integer passed to HAC.variance for `vcov.type = "HAC"`. Must not exceed the shorter fitted regime length. If `NULL`, `max(1, floor(n_j^(1/4)))` is used separately in each regime.
`kernel`	A HAC kernel: `Bartlett`, `Parzen` or `Tukey-Hanning`.
`time.index`	Optional original integer-period positions for HAC lag products, with one entry per input row. Does not change break-date indexing. The estimation sample must be contiguous; internal missing rows are rejected.

Details

For every admissible date b, coefficients are reestimated by OLS. The profile Gaussian quasi-log-likelihood uses ML variances, `RSS_j/n_j` for `regime` and `(RSS_1+RSS_2)/N` for `constant`. The break estimator maximises that same profile. The confidence set contains exactly the dates satisfying $LR(b) = 2[\ell(\hat{b}) - \ell(b)] \leq c_{level}$.

Two SSR.recursive passes from `breaktest` compute all candidate residual sums of squares. OLS.reg supplies the fitted regimes at the maximum. The scales $\omega_i = \Gamma_i^2/\Psi_i$ are estimated once there, following Proposition 1, and held fixed when testing candidate dates. The CDF is $(1-\exp(-c/(2\omega_1)))(1-\exp(-c/(2\omega_2)))$. HAC calculations reuse HAC.variance and the package's kernel functions.

In the univariate notation, let $v_i = \hat{\sigma}_i^2$, $d = \hat{\beta}_2 - \hat{\beta}_1$ and $M_i = X_i'X_i/n_i$. Then $B_1 = (v_2 - v_1)/v_2$, $B_2 = (v_2 - v_1)/v_1$, $Q_1 = M_1/v_2$, $Q_2 = M_2/v_1$ and $\Psi_i = B_i^2/2 + d'Q_id$. The score sequences are X_1u_1/v_2 and X_2u_2/v_1; their covariance estimates give Π_i. For `iid`, these simplify to $\Pi_1 = v_1M_1/v_2^2$ and $\Pi_2 = v_2M_2/v_1^2$. Covariances of $u_i^2/v_i - 1$ give Ω_i, and $\Gamma_i^2 = B_i^2\Omega_i/4 + d'\Pi_id$.

Validity requires the assumptions of the paper, including stabilising moments within regimes (no unit roots or deterministic trends), an identifiable break and the zero third-moment condition used in Proposition

1. This is inference on timing conditional on a break, not a test for its existence. Multiple breaks, multivariate systems, partial coefficient restrictions and a variance-only restricted model are not implemented here. For inference about a coefficient break allowing a variance break, use `variance = "regime"`; this does not constrain the coefficients to be equal.

Value

An object of class `bt_fcstConfset`, a list of:

- `bp`: the profile-likelihood maximiser (earliest in an exact tie),
- `set`: every accepted last-pre-break date,
- `lower`, `upper`: the envelope of the set, not an interval replacing it,
- `intervals`: contiguous components with columns `lower` and `upper`,
- `profile`: all candidate dates, log likelihoods, LR values and acceptance,
- `critical.value`, `level`, `omega`: the cutoff and its calibration,
- `scale`: the estimated terms from Proposition 1,
- `fit`: the two pilot OLS.reg regressions,
- `variance`, `vcov.type`, `bandwidth`, `kernel`, `sigma`, `N`, `k`, `trim`: model settings, pilot standard deviations and sample information.

References

Eo, Yunjong, and James Morley. "Likelihood-Ratio-Based Confidence Sets for the Timing of Structural Breaks." Quantitative Economics 6, no. 2 (2015): 463-497. https://doi.org/10.3982/QE186.

`fcst.DGP` *Reproducible forecast experiments under parameter instability*

Description

The ten mean-instability designs in Section 4 of Hirano-Wright, and an end-of-sample comparison using fcst.compare. Forecasts see only their training history. The future signal and observations are used for scoring.

Usage

```
fcst.DGP(N = 100L, design = 1L, h = 1L, rho = 0, seed = NULL)

fcst.simulate(
  N = 100L,
  design = 1:10,
  n.rep = 100L,
  h = 1L,
  rho = 0,
  methods = c("full", "postbreak", "exps"),
  benchmark = "full",
  method.options = list(),
  forecast.options = list(),
  seed = 123L,
  n.cores = 1L,
  keep.data = FALSE,
  verbose = TRUE
)
```

Arguments

N	Training sample length; a vector in fcst.simulate.
design	Design numbers 1:10; a vector in fcst.simulate. 1: stable; 2:4: a single jump of 10/sqrt(N) after N*(.25,.5,.75); 5:7: random-walk signal innovations with SD (1,5,10)/N; 8:10: compound Poisson jumps with rate (1,2,3)/N per period and jump SD 3/sqrt(N). The pre-sample signal b[0] is zero in every design; a stochastic increment may already occur at observation 1.
h	Positive forecast horizon(s).
rho	AR(1) coefficient of the observation disturbance, |rho|<1. Its innovation SD is 1-rho, so the long-run variance is one. The initial disturbance is drawn from its stationary distribution. A vector is allowed in fcst.simulate.
seed	Optional integer seed. Explicit seeds restore the caller's RNG state; NULL consumes it normally. Simulation uses an independent L'Ecuyer stream per (design,rho,N,replication) task, in fixed grid order.
n.rep	Number of independent replications per design cell.
methods, benchmark, method.options	As in fcst.compare.
forecast.options	Named list of shared forecasting options, e.g. list(lags=1:2, type="direct", trim=.2). No future latent states are passed.
n.cores	Number of SOCK workers. Parallelism is across replications; each inner forecast comparison remains sequential.
keep.data	Retain complete generated paths; FALSE keeps only the targets, losses, jump dates and settings needed for compact reporting.
verbose	Print progress from the main process.

Details

fcst.DGP returns all N+max(h) observations, signal and disturbances, actual jump sizes/counts and dates. Multiple Poisson events in one period are accumulated, not discarded. The signal starts from b[0]=0, as specified in the paper. AR disturbances implement the paper's serial-correlation experiments, which the supplied mcprog.m omits.

The future latent signal b[N+h] is the target of the paper's scaled risk. It is distinct from both observed y[N+h] and E[y[N+h]|b[N],u[N]], which is b[N]+rho^h*u[N] for these end-of-sample designs. All three targets are returned explicitly. Simulated latent states never enter forecast fitting. The stationary AR initialisation makes an arbitrary burn-in unnecessary. Changing N, rho or h defines a new experiment within these DGP formulas; it does not claim to reproduce an unreported table or extend a forecasting method's theoretical domain. Each method retains its own source restrictions.

Value

fcst.DGP returns a bt_fcstDGP; fcst.simulate returns a bt_fcstSimulation with settings, per-replication losses, failures, breaks, summary (MSFE, RMSFE, latent-signal and oracle-mean risk, MC standard errors), conditional.summary, and optional generated data. Unconditional summary risk and MCSE are returned only if every planned replication has valid forecasts from every method at every requested horizon. Otherwise these entries are NA; conditional.summary reports the selected common successes with an explicit scope label. Those conditional risks are not estimates of the original experiment's unconditional risk. Raw losses and failures remain visible. MCSE is across replications, not time points.

References

Hirano, K. and J. H. Wright (2022). "Analyzing Cross-Validation for Forecasting with Structural Instability." Journal of Econometrics 226, 139-154. Section 4 and equation (4.1). https://doi.org/10.1016/j.jeconom.2020.10

`fcst.ExpS` *Data-driven exponential discounting*

Description

Select the exponential weight using rolling-origin CV, Laplace CV, or the local-level maximum-likelihood and trade-off rules of Hirano-Wright. Regression fits reuse OLS.reg and WLS.reg.

Usage

```
fcst.ExpS(
  y,
  x = NULL,
  x.new = NULL,
  method = c("cv", "cvl", "ml", "tradeoff"),
  gamma.grid = NULL,
  eta.grid = NULL,
  omega = NULL,
  prior = c("midpoint", "uniform"),
  sigma.sq = NULL
)
```

Arguments

`y`, `x`, `x.new`	An aligned one-step regression, as in fcst.forecast.
`method`	"cv", "cvl", "ml" or "tradeoff".
`gamma.grid`	Candidate finite-sample discount factors in `[0,1]`.
`eta.grid`	Alternative local-parameter grid: gamma = 1 - eta/T, with T the original history length. Supply only one grid. The default is 301 equally spaced eta values from 0 to min(30, 0.99*T).
`omega`	First inner forecast origin. Defaults to the last 10 percent of the history, as in fcst.window.cv.
`prior`	For CVL, "midpoint" implements the uniform prior on the continuous exponential kernel's median location in `[0.5,0.95]`, as in Section 3.2 of the paper, integrated over grid cells. "uniform" assigns equal mass to equally spaced eta candidates, as in mcprog.m. Irregular grids are rejected for this prior. A truncated grid truncates the prior.
`sigma.sq`	A fixed estimate of the equation-error variance for CVL. By default use the unweighted direct-regression residual mean square. It must estimate the same variance as in the source criterion; it is not an arbitrary temperature parameter. Zero uses the limiting posterior on criterion minima. This is not a posterior from a fully specified likelihood for dependent forecast errors.

Details

CVL averages eta and refits at the resulting gamma. It does not average candidate forecasts. CV scores all candidates on the same usable origins; a candidate failing on any such origin is inadmissible. Exact CV ties prefer the largest gamma. Adaptive methods require a regular retained sample without internal missing periods.

ML and tradeoff require a complete, equally spaced, constant-only model. They fit the exact zero-mean Gaussian MA(1) likelihood of diff(y), with MA coefficient -gamma in `[-1,0]`, the invertible local-level branch. The profile need not be unimodal: all local minima detected on a dense grid, supplemented near the unit-MA boundary, are refined and compared with both boundaries. The reported optimisation diagnostics describe this deterministic numerical search; it is not a proof of uniqueness. Unlike unconstrained MATLAB optimisation, a positive MA coefficient cannot create negative weights. The fitted variances are sigma.error^2 = gamma*sigma.MA^2 and sigma.level^2 = (1-gamma)*^2sigma.MA^2. The local magnitude for (A.6) is mu = T*sigma.level/sigma.error;* *T*(1-gamma) in MATLAB is its local approximation. The trade-off rule minimises (A.6) on eta in `[0,T]`.

For direct multiple horizons use fcst.forecast with method="exps" and gamma.method set to one of these methods. ML/tradeoff support h=1 or iterated constant-level forecasts; direct h>1 and nonconstant regressions require CV/CVL. Iterated CV/CVL for h>1 and nested predictor selection are not part of the source procedure.

Value

A bt_fcstExpS with forecast, coefficients, normalised weights, gamma, eta, method, fit, and the CV profile/posterior or ML diagnostics.

References

Hirano, K. and J. H. Wright (2022). "Analyzing Cross-Validation for Forecasting with Structural Instability." Journal of Econometrics 226, 139-154. Sections 2.3, 3.2, 4 and equation (A.6). https://doi.org/10.1016/j.jeconom.2020.10.009.

`fcst.FGLS` *Two-step break inference and forecasting of Altansukh and Osborn*

Description

Detects a coefficient break and a possibly distinct variance break, then repeats coefficient inference after feasible GLS transformation. Uses fcst.test.Wald and the package's OLS.reg.

Usage

```
fcst.FGLS(
  y,
  x = NULL,
  x.new = NULL,
  trim = 0.1,
  level = 0.05,
  inference = c("two.step", "HC"),
  stepwise = FALSE,
  step.level = level,
```

```
  hc.type = c("HC0", "HC3"),
  average = FALSE,
  conf.method = c("eo.morley", "bai"),
  conf.level = 0.95,
  time.index = NULL
)
```

Arguments

y, x, x.new	An aligned regression and one forecast row.
trim	Symmetric trimming; .1 is the paper's setting.
level	Nominal asymptotic level of the sup-Wald tests.
inference	"two.step" (FGLS) or the paper's "HC" comparator.
stepwise	Test individual coefficient changes after detecting a break.
step.level	Nominal level for the individual asymptotic two-sided t tests. Remove the least significant change and refit until all remaining changes are significant. These are conditional selection decisions, not multiplicity-adjusted or post-selection p-values.
hc.type	HC covariance for the pilot and HC comparator: HC0 or HC3.
average	Average forecasts over a confidence set for the coefficient break after the final stability test rejects. The variance transformation is held fixed and coefficient selection is repeated at every accepted date.
conf.method	Confidence set method: "eo.morley" or "bai".
conf.level	Confidence probability, distinct from the test size level.
time.index	Original integer-period positions of observations. Internal gaps in the estimation block are not supported.

Details

The pilot is a robust coefficient test. Its least-squares date is used only if the test rejects. Fit that mean model and test sqrt(pi/2)*abs(residual) for a constant-mean break using homoskedastic covariance. If detected, the two regime means estimate standard deviations (not variances). Divide y and every x column by these deviations, and repeat the coefficient test with homoskedastic covariance. There is no iteration of the variance step. The forecast uses the final regime's coefficients, with stable coefficients estimated over the full sample when stepwise=TRUE. Without a detected coefficient break it uses full-sample OLS or FGLS as appropriate.

Dates index complete regression rows and denote the last pre-break response. With average=TRUE, the confidence set is computed on the original data, allowing variance and regressor moments to change across coefficient regimes (Section 2.3). It is not computed on the FGLS-transformed sample. Each date in this set gives a separate OLS/FGLS and optional stepwise fit; forecasts receive equal weights, preserving disconnected sets. The last window start cannot exceed N-trim*N, as in Section 2.1. Failure of a component fit is an error, rather than a silent change to the averaging measure. fcst.CIavg remains the separate, unconditional raw-data OLS combination. The source assumes serially uncorrelated innovations; a variance break does not substitute for a serial-correlation model. The estimation sample must be contiguous after trimming leading or trailing missing observations. Internal missing rows and external selection of the regressor set are not covered by this procedure. The published stepwise restrictions on changes of given coefficients remain available.

Value

A bt_fcstFGLS with forecast, coefficients, bp, variance.bp, transformation scales, all three tests, and the stepwise trace. Averaging adds the confidence set, accepted dates, component coefficients, forecasts, and stepwise traces. `fit` describes the single estimated-date fit; it is not a regression representation of the averaged forecast.

References

Altansukh and Osborn (2022), Empirical Economics 63, 1-41, Sections 2.1-2.3. https://doi.org/10.1007/s00181-021-02137-w.

`fcst.forecast` *Direct and iterated forecasts under structural breaks*

Description

A common interface to all estimation-window, weighting and combination strategies. Fits use the existing regression and break-dating routines.

Usage

```
fcst.forecast(
  y,
  x = NULL,
  x.new = NULL,
 method = c("full", "postbreak", "window", "exps", "ppp", "ppp.robust", "avew", "avelr",
    "ciavg", "stein", "bp.combine", "wgls", "fgls", "wa", "pooled"),
  window.method = "cv",
  bp = NULL,
  trim = 0.15,
  ...,
  h = 1,
  type = c("direct", "iterated"),
  lags = NULL,
  constant = TRUE,
  x.future = NULL,
  index = NULL,
  selection = NULL
)
```

Arguments

`y`	A univariate series on a regular observation grid. Keep missing periods as NA rows; lags and horizons count original rows, not complete cases.
`x`	With lags = NULL, an already aligned design: row t explains `y[t]` and must be available at t-1. Include any constant explicitly. With explicit lags, external predictors observed at their row's date; the generated one-step design uses `x[t-1]`. NULL means no external predictors.
`x.new`	One forecast-design row available at the current origin, for the already aligned input. Required for h > 1 when x is supplied. The legacy h = 1 default is the last complete design row. Do not supply with lags.

`method`	One of full, postbreak, window, exps, ppp, ppp.robust, wgls, avew, avelr, ciavg, stein, bp.combine, fgls, wa, pooled.
`window.method`	A rule in fcst.window.
`bp`	Last pre-break response row. With explicit lags or h > 1, indexes the original y; the same applies with predictor selection enabled. It is mapped to the horizon-specific training sample. Legacy h = 1 matrix calls retain complete-case indexing. NULL estimates it. FGLS, WA and pooled always use original-row dates. FGLS estimates its own dates, so bp must be NULL for that method.
`trim`	A break-date trimming fraction in (0, 0.5).
`...`	Method-specific arguments. WGLS uses the published post-break LOO criterion at every horizon; cv.method="loo" is its only supported value. ExpS accepts gamma.method="fixed" (default), "cv", "cvl", "ml" or "tradeoff" and the corresponding fcst.ExpS options. Supplied CIavg set/interval use the same date convention as bp.
`h`	Positive integer horizon(s). A scalar returns a single forecast; a vector returns results for each distinct horizon.
`type`	"direct" fits a separate h-step regression; "iterated" estimates a one-step rule and recursively applies its fixed coefficients.
`lags`	NULL preserves the supplied-design interface. An explicit vector, e.g. 1:2, constructs `y[t-1]`, `y[t-2]` automatically. integer(0) selects the generated design without AR terms. Required for iterated h > 1 with nonconstant x.
`constant`	Add a constant in the generated design. Ignored when lags is NULL.
`x.future`	External x at T+1,...,T+h-1 for iteration. Supply a matrix or a function function(origin, h, y, x) returning those rows. The function receives only histories through origin and is required for inner CV and fcst.compare with external predictors. Direct forecasts do not use it.
`index`	Optional original row labels, defaulting to time(y) for ts and row numbers otherwise. Labels do not insert missing periods.
`selection`	NULL disables predictor selection. A named list of fcst.select options (always, p, delta, vcov.type, bandwidth) enables unweighted single-stage OCMT on each training history. Supported downstream strategies are full, fixed-gamma exps, ppp and ppp.robust. Other combinations and predictor selection inside tuning CV are not implemented. Returned coefficients retain all original columns, with zeros for unselected ones.

Details

The generated direct regression pairs `y[t+h]` with a constant, `y[t+1-lags]` and `x[t]`, using only pairs with t+h <= T. An already aligned matrix instead pairs design row r with `y[r+h-1]`. No future realised y enters an iteration. Window/weight selection is frozen at the real origin. WA, pooled and FGLS confidence-set averaging use individual recursive paths. The current AveW/AveLR/CIavg and Boot-Pick combinations are direct-only for h > 1.

IJR and adaptive window/ExpS CV/CVL support direct h-step regressions, not nonlinear iterated tuning. FGLS supports h=1 and iterated forecasts; its residual-variance procedure is not applied to overlapping direct errors. Unsupported method/horizon/selection combinations fail explicitly.

Stein uses its published iid/FGLS coefficient estimate, never a HAC-modified shrinkage rule. Direct Eo-Morley/Bai and Boot-Pick use HAC by default at h>1. Source assumptions remain necessary for inference; no blanket risk dominance is claimed. Simple recursion of a fixed estimate is a plug-in baseline. Statistical methods require contiguous complete training observations. Full OLS, fixed

ExpS and a supplied post-break window retain descriptive complete-case forecasts. These are point forecasts, not prediction intervals.

Value

A bt_fcst with forecast, coefficients, weights, bp, details, h, type, origin and target. Generated/multi-step calls also return training (original origin/target row map), bp.row (internal complete-case break index), and the recursive path for iteration. With multiple horizons a bt_fcstMulti contains forecasts and by.horizon, retaining all individual results.

References

Marcellino, M., J. H. Stock and M. W. Watson (2006). "A Comparison of Direct and Iterated Multistep AR Methods for Forecasting Macroeconomic Time Series." Journal of Econometrics 135, 499-526. https://doi.org/10.1016/j.jeconom.2005.07.020.

---

fcst.select *One-covariate-at-a-time selection before forecast estimation*

Description

The single-stage OCMT procedure of Chudik, Pesaran and Sharifvaghefi: regress y on each candidate and the always-kept conditioning variables, without observation down-weighting. Retain candidates with absolute t-ratios above the multiple-testing threshold.

Usage

```
fcst.select(
  y,
  x,
  always = NULL,
  p = 0.1,
  delta = 1,
  vcov.type = c("const", "HC0", "HC3", "HAC"),
  bandwidth = NULL,
  time.index = NULL
)
```

Arguments

y	A numeric univariate response.
x	A numeric design, including any constant explicitly. The number of candidates may exceed the number of observations.
always	Conditioning columns, by index or unique name. NULL keeps nonzero constant columns. integer(0) specifies no conditioning variables.
p	Nominal multiple-testing size in (0, 1).
delta	Exponent in qnorm(1 - p / (2 * n.candidates^delta)). Must be at least 1 for const and strictly greater than 1 for robust covariance. An omitted delta defaults to 1 for const and 1.1 otherwise; delta.source records which default, or the user's explicit value, was used.

`vcov.type` "const" uses RSS/N, as in the paper. "HC0", "HC3" and "HAC" use the module's sandwich covariance. Robust standard errors with delta > 1 are explicitly proposed in the source's Section 5.2, footnote 3.

`bandwidth, time.index` HAC settings as in HAC.variance. Original integer time positions are retained after complete-case filtering.

Details

This is the single-stage procedure in Section 3, equations (4)-(5), of the parameter-instability paper, which excludes hidden signals. It is not the multi-stage OCMT for hidden signals in Chudik et al. (2018), nor the grouped Ibragimov-Muller test mentioned in comments in ac/application_*.m. The paper's normal critical value replaces MATLAB's t quantile with 10000 degrees of freedom. HAC robustness does not itself establish the paper's variable-selection guarantees under an arbitrary dependence process. The source's signal separation, conditioning-set, exogeneity, tail and dependence assumptions remain necessary. The robust recommendation is not a general high-dimensional HAC theorem. Selection is unweighted; subsequent down-weighted forecast estimation has direct support in Section 6.2. Repeating OCMT inside other named adaptive procedures needs separate support and is restricted by fcst.forecast.

All marginal regressions use the same complete cases. Collinearity with the conditioning set is reported, not silently treated as a significant coefficient. The selected joint design can still be rank deficient; its rank is returned and the forecast interface refuses unidentified fits. Retained observations must be consecutive on the original time grid. Leading/trailing incomplete rows may be trimmed; internal gaps are refused.

Value

A bt_fcstSelect with selected column indices/names, conditioning and candidate columns, the full table of t-ratios, threshold and reasons, complete-case rows, and the rank of the selected joint design.

References

Chudik, A., M. H. Pesaran and M. Sharifvaghefi (2024). "Variable selection in high dimensional linear regressions with parameter instability." Journal of Econometrics 246, 105900. Section 3, equations (4)-(5), and Section 5.2, footnote 3, of the July 2024 revision (Dallas Fed Globalization Institute Working Paper 394, version 3; cover dated August 2024) (earlier title: "Variable Selection and Forecasting in High Dimensional Linear Regressions with Parameter Instability"). https://doi.org/10.1016/j.jeconom.2024.105900. https://www.dallasfed.org/~/media/documents/research/international/w

---

`fcst.stein` *Stein-like combination under the published iid GLS model*

Description

Combine the full-sample estimate and the estimate after a structural break, using alpha = min(1, tau/H) as in Lee, Parsaeian and Ullah. The implementation is restricted to a single coefficient break and uncorrelated errors with constant variance within each regime.

Usage

```
fcst.stein(
  y,
  x = NULL,
  x.new = NULL,
  bp = NULL,
  tau = NULL,
  trim = 0.15,
  gls = TRUE,
  vcov.type = "iid",
  bandwidth = NULL,
  kernel = NULL,
  time.index = NULL
)
```

Arguments

y, x, x.new	An aligned regression as in fcst.forecast.
bp	Last pre-break observation in the complete-case sample. NULL estimates a single break with fcst.window.LS. The model must have only one actual break; supplying the last of several breaks is not supported.
tau	Nonnegative shrinkage parameter. Default max(k-2,0).
trim	Trimming for estimation of an unknown break date.
gls	Must be TRUE: use inverse regime-variance weights in the full-sample fit. The former OLS extension is not implemented.
vcov.type	Must be "iid". The former HAC shrinkage extension has been removed because the published risk rule does not justify it.
bandwidth, kernel	Unsupported legacy HAC settings; must be NULL.
time.index	Original integer-period positions. The estimation block must be contiguous after trimming leading or trailing missing rows.

Details

Write B_F = (X'aX)^(-1), B_P = (X_P'X_P)^(-1), with scalar pilot weights a_t. Rows of the linear maps are L_F[t,] = a_t*x_t'*B_F and L_P[t,] = 1(t>bp)*x_t'*B_P. The error covariance is diagonal with regime variances estimated from regime OLS residuals. Applying these linear maps gives coefficient covariance blocks V_F, V_P and C=Cov(beta_F,beta_P). Then

$$V_D = V_P + V_F - C - C', \qquad H = (\hat{\beta}_P - \hat{\beta}_F)' V_D^{-1} (\hat{\beta}_P - \hat{\beta}_F).$$

No further factor N belongs in H. With GLS the difference covariance reduces to V_D=V_P-V_F.

The source paper assumes uncorrelated errors within and across regimes. The risk.eligible flag checks only dimension and the range of tau in the source result. Its model, estimation and quadratic-loss conditions W=V_D^(-1) must also hold. It does not assert lower MSFE for an arbitrary forecast direction, or verify the assumptions from observed data. tau=0 returns the post-break fit when the method is identified. A singular difference covariance produces an error, without a fallback strategy.

Value

A bt_fcstStein with forecast, coefficients, alpha, H, tau, beta.full, beta.post, forecast.full, forecast.post, regime sigma, vcov (joint/full/post/cross/difference), covariance.rank, `risk.eligible` and the conditional interpretation `risk.conditions`. No unconditional `dominance` result is returned.

References

Lee, T.-H., S. Parsaeian and A. Ullah (2022). "Optimal Forecast under Structural Breaks." Journal of Applied Econometrics 37, 965-987. Equations (4)-(6), Theorem 1 and Corollary 3.1. https://doi.org/10.1002/jae.2908.

`fcst.test.BP` *Test whether modelling a structural break improves forecast accuracy*

Description

Boot and Pick (2020) point out that the classical break tests answer the wrong question. Under the MSFE loss there is a bias-variance trade-off, so a break that is statistically detectable may still be too small to be worth modelling. Their null hypothesis is therefore *equal forecast accuracy* between the post-break (or the combined) forecast and the full-sample forecast, which maps into a non-zero critical break magnitude.

Two features distinguish the resulting statistic from the sup-Wald of Andrews (1993):

- the coefficient difference is weighted by the regressors of the forecast origin, so a large break in a coefficient can be irrelevant for the forecast while a small one can be decisive;
- under a known break date the statistic is $\chi^2(1,1)$ rather than $\chi^2(1)$, and the 5% critical value is 7.00 instead of 3.84.

Usage

```
fcst.test.BP(
  y,
  x = NULL,
  x.new = NULL,
  trim = 0.15,
  level = 0.05,
  target = c("postbreak", "combined"),
  break.vars = NULL,
  vcov.type = "HC3",
  cv.table = NULL,
  bandwidth = NULL,
  time.index = NULL
)
```

Arguments

`y`	A dependent variable.
`x`	Explanatory variables. If `NULL` a single constant is used.
`x.new`	A (1 x k) vector of regressor values for the forecast origin. Defaults to the last row of `x`.

`trim`	A trimming fraction bounding the admissible break dates.
`level`	A nominal size, one of 0.01, 0.05 or 0.10.
`target`	Which comparison is tested: • `postbreak`: post-break forecast versus full sample, • `combined`: the combined forecast versus full sample.
`break.vars`	Indices of the columns of `x` subject to the break. `NULL` means a pure structural change in all the coefficients. The empirical part of the original paper allows a break in the intercept only, which corresponds to `break.vars = 1`.
`vcov.type`	One of `const`, `HC0`, `HC3` or `HAC`. The original code uses `HC3` in its heteroskedasticity robust mode.
`cv.table`	An optional replacement for the built-in critical values, either the full output of fcst.BP.simulate or its target-specific branch. Custom branches require metadata with `trim` and `target` fields.
`bandwidth`	Optional HAC bandwidth (products through bandwidth-1).
`time.index`	Original integer-period positions. The estimation sample must be a contiguous block; internal missing observations are unsupported.

Details

The statistic is

$$\sup_{\tau} W(\tau) = \sup_{\tau} \frac{\left[x'_{N+1}(\hat{\beta}_1(\tau) - \hat{\beta}_2(\tau))\right]^2}{x'_{N+1}\widehat{Var}(\hat{\beta}_1(\tau) - \hat{\beta}_2(\tau))x_{N+1}},$$

and the size-corrected version is $S(\hat{\tau}) = \sqrt{W(\hat{\tau})} - \zeta^{1/2}(\hat{\tau})$, whose critical values are almost independent of the break date.

The combined forecast reproduces the optimally weighted forecast of Pesaran et al. (2013) written as a convex combination:

$$\hat{y}^c = \frac{1}{1+W}\hat{y}^{full} + \frac{W}{1+W}\hat{y}^{post}.$$

The entire requested trimming region must be identified. Singular split designs, nonpositive projected variances or sample-size restrictions that shorten that region produce an error, rather than a different search rule. Partial changes and consistent HC/HAC covariance estimates are supported under the paper's moment and functional-limit assumptions. These assumptions are not established by the covariance option or the table compatibility flag.

Value

An object of class `bt_fcstBP`, a list of:

- `W`, `S`: the two test statistics evaluated at the estimated break date,
- `cv.W`, `cv.S`, `zeta.sqrt`: critical values and the equal performance boundary at the estimated break date,
- `reject.W`, `reject.S`: test decisions,
- `bp`, `tau`: the estimated break date and fraction,
- `forecast`: a named vector with the `full`, `postbreak` and `combined` point forecasts,
- `weight.full`: the weight $1/(1+W)$ placed on the full-sample forecast in the combination,
- `W.all`: the whole profile of the statistic over the admissible dates.
- `calibrated`, `inference`: whether the table matches the search region and a diagnostic. Unsupported calibration retains point forecasts and W, but S, critical values and test decisions are NA.

References

Boot, Tom, and Andreas Pick. "Does Modeling a Structural Break Improve Forecast Accuracy?" Journal of Econometrics 215 (2020): 35-59. Local author version: 2019. https://doi.org/10.1016/j.jeconom.2019.07.007.

`fcst.test.Wald` *Sup-Wald and exponential-Wald structural change tests*

Description

Tests a common unknown break in selected regression coefficients. Each candidate date is fitted with OLS.reg and the module's covariance routines. The statistic is in chi-square units, not divided by the number of tested coefficients. Approximate p-values use Hansen's (1997) response surfaces.

Usage

```
fcst.test.Wald(
  y,
  x = NULL,
  break.vars = NULL,
  trim = 0.15,
  trim.exp = trim,
  vcov.type = "HC0",
  bandwidth = NULL,
  time.index = NULL,
  level = 0.05
)
```

Arguments

`y`, `x`	An aligned regression; NULL x supplies a constant.
`break.vars`	Original design columns allowed to change. NULL tests all.
`trim`, `trim.exp`	Symmetric trimming fractions for sup and exp tests. Dates strictly inside (trim*N, (1-trim)*N) are searched. The effective endpoints are reported and used in the p-value approximation.
`vcov.type`	"const", "HC0", "HC3" or "HAC". HC0 is the default.
`bandwidth`, `time.index`	HAC bandwidth and original integer positions. The estimation block must be contiguous; internal gaps are unsupported.
`level`	Nominal rejection probability.

Details

exp = log(mean(exp(W(date)/2))) is evaluated by log-sum-exp. The restricted coefficients remain constant throughout the sample. All admissible dates must be identified; singular dates are not silently removed from the test's search region. bp maximizes Wald; bp.LS minimizes the unrestricted SSR. They need not coincide with robust covariance. The sup and exp p-values require the source asymptotic assumptions and are not exact finite-sample probabilities under arbitrary nonstationarity or post-selection.

Value

A bt_fcstWald with named statistics, p.value, critical.value, reject, estimated dates, effective trimming, and the complete profile. `p.value.status` marks p-values outside the monotone response-surface range. These are `NA`; decisions still compare the statistic to an available critical value at the requested level.

References

Andrews (1993), Econometrica 61, 821-856, and Andrews-Ploberger (1994), Econometrica 62, 1383-1414. Hansen (1997), "Approximate Asymptotic P Values for Structural-Change Tests", JBES 15, 60-67. https://users.ssc.wisc.edu/~behansen/progs/jbes_97.html.

`fcst.WA` *Pesaran-Timmermann weighted and pooled window forecasts*

Description

Average expanding-window forecasts, either with weights inverse to past MSFE (WA) or equal weights (pooled). An optional single coefficient-break pretest chooses between the full sample and break-restricted averaging.

Usage

```
fcst.WA(
  y,
  x = NULL,
  x.new = NULL,
  bp = NULL,
  trim = 0.15,
  min.window = NULL,
  omega = NULL,
  pretest = FALSE,
  level = 0.05,
  vcov.type = NULL,
  bandwidth = NULL,
  ...
)

fcst.Pooled(
  y,
  x = NULL,
  x.new = NULL,
  bp = NULL,
  trim = 0.15,
  min.window = NULL,
  pretest = FALSE,
  level = 0.05,
  vcov.type = NULL,
  bandwidth = NULL,
  ...
)
```

Arguments

y, x, x.new	Regression inputs, as in fcst.forecast.
bp	Known last pre-break response row on the original grid. Windows start no later than bp+1. NULL with pretest=FALSE averages all candidates.
trim	Trimming for an unknown-date pretest and default minimum window.
min.window	Minimum estimation length is strictly greater than this integer (omega in the paper). Default max(k,round(trim*N)).
omega	First inner-CV origin (T minus omega-tilde in the paper), default round(.75*T). Only WA uses this setting. Each validation target must already have been observed at the outer forecast origin.
pretest	Run fcst.test.Wald on the outer training sample. Failure to reject gives the full-sample forecast; rejection restricts starts to at most bp.LS+1. Cannot be combined with a supplied bp.
level, vcov.type, bandwidth	Pretest settings; NULL covariance chooses HC0, or HAC for direct h>1. This does not establish post-selection validity.
...	Horizon, lags, selection and other common fcst.forecast settings.

Details

WA computes an ordinary recursive forecast error for each start and each common validation origin. Weights are proportional to 1/MSFE. The prose of Pesaran-Timmermann Section 3.4 and Altansukh-Osborn Appendix A specify inverse weights; the printed numerators in PT (22)-(23) omit the inverse. Here the inverse is used. Zero-error candidates share all mass; a candidate failing any usable fold has zero weight and its failure is recorded. The paper's bound start<=T-omega-omega-tilde ensures all validation fits exceed omega.

Pooled averaging uses equal weights and the bound start<=T-omega. Its break-restricted variant is the forecast average in PT Section 3.5, not a single regression fitted to pooled observations. Observation weights are descriptive inclusion weights, not equivalent regression weights.

Predictors must be specified before fitting; nested OCMT selection is not part of this source procedure. The optional outer pretest conditions the candidate set on information at the forecast origin, as in the paper; inner errors are not an unbiased evaluation of the entire pretest strategy. fcst.compare supplies that outer evaluation. For iterated h>1, each window's recursive path is computed separately and the paths are averaged with fixed weights. The returned coefficient vector is a weighted summary and need not generate that averaged recursive path.

Value

A bt_fcst with window forecasts, weights, validation errors, selection diagnostics and pretest in details.

References

Pesaran and Timmermann (2007), Journal of Econometrics 137, 134-161, Sections 3.3-3.6. https://doi.org/10.1016/j.jeco
Altansukh and Osborn (2022), Appendix A.

`fcst.WGLS.cv` *Weighted GLS with post-break leave-one-out cross-validation*

Description

The feasible WGLS forecast of Lee, Parsaeian and Ullah (2022), equations (11)-(13). The Aitchison-Aitken kernel discounts pre-break observations by `gamma`; accounting for the regime variances gives the effective relative weight $\gamma^* = \gamma/q^2$, where $q = \sigma_{(1)}/\sigma_{(2)}$ is a standard deviation ratio.

Usage

```
fcst.WGLS.cv(
  y,
  x = NULL,
  x.new = NULL,
  bp = NULL,
  trim = 0.15,
  gamma.grid = seq(0, 1, by = 0.01),
  gamma.star.grid = NULL,
  time.index = NULL
)
```

Arguments

`y`	A dependent variable. Incomplete rows of `y` and `x` are removed jointly. The retained observations must form a regular uninterrupted sample.
`x`	Explanatory variables. If `NULL` a single constant is used.
`x.new`	A (1 x k) vector of regressors for the forecast. Defaults to the last complete row of `x`.
`bp`	The last pre-break observation in the complete-case sample. If `NULL` the single break is estimated by fcst.window.LS. Both regimes must have full column rank and more observations than regressors.
`trim`	A trimming fraction in (0, 0.5), used when `bp` is estimated.
`gamma.grid`	Candidate kernel parameters in `[0, 1]`. By default the grid is `seq(0, 1, by = 0.01)`, as in `ac/empirfunc.m`. Zero is included as the continuous post-break-only limit of the paper's kernel.
`gamma.star.grid`	Optional finite, nonnegative effective weights. When supplied, implements the direct search in Remark 3 without estimating `q`. Values above one are allowed. Do not also supply `gamma.grid`.
`time.index`	Original increasing integer observation positions.

Details

By default, regime OLS residuals give $\hat{\sigma}_j^2 = RSS_j/n_j$ and $\hat{q}^2 = \hat{\sigma}_1^2/\hat{\sigma}_2^2$. These pilot estimates and the break date are held fixed throughout cross-validation. Each post-break row is omitted in turn from the weighted regression, while all pre-break rows remain. The criterion returned is the mean squared LOO error. Dividing it by the fixed post-break variance yields equation (13) and leaves its minimiser unchanged. Direct search needs no variance scale.

For fixed weights the LOO error is $e_s/(1-h_s)$, where $h_s = w_s x_s'(X'WX)^{-1}x_s$. Thus only one WLS.reg fit per candidate is needed. Candidates with singular or numerically unstable fits, including any post-break deletion with leverage near one, receive `Inf`. An error is raised if no candidate is valid. Grids are sorted and deduplicated; exact ties select the smaller effective weight.

This low-level routine takes an already aligned regression. Multi-step direct and fixed-coefficient iterated forecasts use the same source LOO through fcst.forecast; rolling-refit CV is not supported. In the two-step version `gamma = 1` gives full-sample GLS; it gives OLS only when the estimated regime variances coincide. `gamma.star = 1` always gives OLS. The finite search grid in direct mode is chosen by the caller: Remark 3 does not prescribe an upper bound, and `gamma.star` need not be below one.

Value

An object of class `bt_fcstWGLS`, a list of:

- `forecast`, `coefficients`, `weights`: the prediction, fitted coefficients and observation weights normalised to sum to one,
- `bp`, `N`, `k`: the break date and complete-case sample dimensions,
- `gamma`, `gamma.star`: selected kernel and effective weights (`gamma` is `NA` in direct mode),
- `q`, `sigma`: pilot standard deviation ratio and named regime standard deviations (`NULL` in direct mode),
- `cv`: a data frame of `gamma`, `gamma.star` and the unscaled `criterion`,
- `cv.min`, `loo.errors`: the selected criterion and its post-break errors,
- `fit`: the WLS.reg fit on all observations at the selected weight. The adaptive `fit` retains residual diagnostics, but its covariance, standard errors and t-statistics are unavailable: fixed-weight inference does not account for selection of the break date or weight.

References

Lee, Tae-Hwy, Shahnaz Parsaeian, and Aman Ullah. "Forecasting under Structural Breaks Using Improved Weighted Estimation." Oxford Bulletin of Economics and Statistics, 2022.

`get.cv.KY` *Response-surface critical values for cset_break() (Kurozumi & Yamamoto, 2015).*

Description

Response-surface critical values for cset_break() (Kurozumi & Yamamoto, 2015).

Usage

```
get.cv.KY(L1, c_level, k_1)
```

Arguments

`L1`	Numeric scalar. Candidate break fraction, T1 / T_y, in (0, 1).
`c_level`	Numeric, 0.90 or 0.95. Confidence level.
`k_1`	Integer, 1 to 9. Number of regressors (columns of `x`) whose coefficient sustains the break.

Value

Named list with four numeric scalars:

cval_em	Critical value for the Elliott-Muller / modified-EM tests.
cval_sup	Critical value for the sup-test.
cval_avg	Critical value for the avg-test.
cval_exp	Critical value for the exp-test.

---

get.pv.Wald *Hansen p-values and critical values for structural change tests*

Description

Hansen p-values and critical values for structural change tests

Usage

```
get.pv.Wald(statistic, df = 1L, trim = 0.15, type = c("sup", "exp"))

get.cv.Wald(df = 1L, trim = 0.15, level = 0.05, type = c("sup", "exp"))
```

Arguments

statistic	A nonnegative sup-Wald or log-mean-exp(Wald/2) statistic.
df	Number of tested coefficients, 1:40.
trim	Symmetric trimming in [.01, .5]. For asymmetric endpoints (a,b), use 1/(1+sqrt(b*(1-a)/(a*(1-b)))).
type	"sup" or "exp".
level	Upper-tail probability for a critical value.

Details

Numerical response-surface tables are verified against both the local GAUSS source and Hansen's author archive. Critical values invert the same p-value function, avoiding typos in the local qa_crit/ap_crit tables. At trim=.5 there is one date: sup is chi-square and exp is chi-square/2. The latter factor corrects the degenerate-date branch in the old code. Beyond the maximum of a relevant concave quadratic response surface, the approximation would reverse the tail. P-values there are NA, not a capped probability. Critical values are inverted only within the monotone region and are NA if the requested tail cannot be bracketed.

Value

A numeric p-value vector or scalar approximate critical value; NA denotes a request outside the usable response-surface range.

`info.criterions` *Information criterions*

Description

Information criterions

Usage

```
info.criterions(obj, criterion, alpha = NULL, y = NULL)

## S3 method for class 'bt_ols'
AIC(obj, k = 2, alpha = NULL, y = NULL)

## S3 method for class 'bt_ols'
BIC(obj, alpha = NULL, y = NULL)

## S3 method for class 'bt_ols'
HQIC(obj, alpha = NULL, y = NULL)

## S3 method for class 'bt_ols'
LWZ(obj, alpha = NULL, y = NULL)
```

Arguments

`obj`	An object of one of the class `bt_ols`. It should be pointed that `AR.reg` and `DOLS.reg` return object that belong to subclasses of `bt_ols`, so they also can be used.
`criterion`	One of the following string values (case insensitive): • `"AIC"`, • `"BIC"`, • `"HQIC"`, • `"LWZ"`.
`alpha`	The coefficient α of y_{t-1} in ADF model. Needed only for criterion modification purposes, see Ng and Perron (2001) for further information. If NULL then no modification is applied.
`y`	The vector of y_{t-1} in ADF model. Needed only for criterion modification purposes.

Details

Calculating the value of the following informational criterions:

- Akaike,
- Schwarz (Bayesian),
- Hannan-Quinn,
- Liu et al.

Value

The value of the desired information criterion.

References

Ng, Serena, and Pierre Perron. "Lag Length Selection and the Construction of Unit Root Tests with Good Size and Power." Econometrica 69, no. 6 (2001): 1519–54. https://doi.org/10.1111/1468-0262.00256.

`KP.seq.statistic` *Sequential statistic for breaks at unknown date.*

Description

Sequential statistic for breaks at unknown date.

Usage

```
KP.seq.statistic(
  y,
  const = FALSE,
  breaks = 1,
  criterion = "aic",
  trim = 0.15,
  max.lag = trunc(12 * (length(y)/100)^(1/4))
)
```

Arguments

`y`	A time series of interest.
`const`	Allowing the break in constant.
`breaks`	A number of breaks.
`criterion`	Needed information criterion: aic, bic, hq or lwz.
`trim`	A trimming value for a possible break date bounds.
`max.lag`	The maximum possible lag in the model.

Details

This procedure is based on ideas of Perron & Yabu (2009).

Value

An estimated Wald statistic.

References

Kejriwal, Mohitosh, and Pierre Perron. "A Sequential Procedure to Determine the Number of Breaks in Trend with an Integrated or Stationary Noise Component: Determination of Number of Breaks in Trend." Journal of Time Series Analysis 31, no. 5 (September 2010): 305–28. https://doi.org/10.1111/j.1467-9892.2010.00666.x.

Skrobotov, Anton. "Survey on structural breaks and unit root tests." Applied Econometrics 58 (2020): 96–141. https://doi.org/10.22394/1993-7601-2020-58-96-141.

`MDF.1br` *MDF procedure for a single unknown break.*

Description

MDF procedure for a single unknown break.

Usage

```
MDF.1br(y, const = FALSE, trend = FALSE, trim = 0.15, criterion = "bic")
```

Arguments

`y`	time series of interest.
`const`	Whether the constant term is allowed to change (Structural change in intercept).
`trend`	Whether the trend term is allowed to change (Structural change in slope).
`trim`	Trimming value for a possible break date bounds.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

Value

A list of sublists each containing

- The value of statistic: $MDF - GLS$, $MDF - OLS$,
- The asymptotic critical values. UR values are included as well.

MDF.mlt *MDF procedure for multiple unknown breaks.*

Description

MDF procedure for multiple unknown breaks.

Usage

```
MDF.mlt(y, const = FALSE, breaks = 1, breaks.star = 1, trim = 0.15, ZA = FALSE)
```

Arguments

y	a time series of interest.
const	Whether the constant term should be included.
breaks	Number of breaks.
breaks.star	Number of breaks got from the Kejrival-Perron procedure.
trim	Trimming value for a possible break date bounds.
ZA	Whether ZA variant should be used.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

Value

A list of sublists each containing

- The value of statistic: $MDF - GLS$, $MDF - OLS$,
- The asymptotic critical values. UR values are included as well.

plot_bubble *Plotting break dates for bubbles*

Description

Plotting break dates for bubbles

Usage

```
plot_bubble(
  y,
  trim = 0.1,
  break_points = NULL,
  plot_type = c("paper", "presentation"),
  date_first = NULL,
  date_format = "%Y-%m",
  date_by = "month",
  y_label = NULL,
  title = NULL
)
```

Arguments

y	Time series of interest.
trim	Trimming parameter for breakpoints estimation procedure.
break_points	Breakpoints to be plotted; if NULL (default) then these points are estimated. If date_first is not NULL break_points should bu numeric or Date, otherwise it should be numeric only.
plot_type	Select the style of the resulting graph.
date_first	If not NULL then the X-axis will be labeled with dates.
date_format	Format of dates on the graph.
date_by	Time delta of your data.
y_label	An optional label for Y-axis.
title	An optional title of the graph.

References

Kurozumi, Eiji, and Anton Skrobotov. 2026. "Confidence Sets for the Emergence, Collapse, and Recovery Dates of a Bubble". arXiv:2511.16172. Preprint, arXiv. https://doi.org/10.48550/arXiv.2511.16172.

---

plot_bubble_confsets *Plotting confidence sets for bubble dates as shaded bands*

Description

Plot a time series with separate shaded components of confidence sets for the emergence, collapse and recovery dates of a bubble.

Usage

```
plot_bubble_confsets(
  y,
  conf_sets,
  break_points = NULL,
  set_index = NULL,
  time = NULL,
  plot_type = c("paper", "presentation"),
  date_first = NULL,
  date_format = "%Y-%m",
  date_by = "month",
  colours = c(Te = "#F4B6C2", Tc = "#6DEB6B", Tr = "#8277F5"),
  conf_alpha = 0.4,
  shade_phases = FALSE,
  phase_alpha = 0.08,
  y_label = NULL,
  title = NULL
)
```

Arguments

y	Numeric time series or a one-column matrix. Missing observations are allowed and retain their original positions.
conf_sets	Named list of binary or logical membership vectors, named Te, Tc and Tr. Select one already computed method per date. One-column membership matrices are also accepted; missing membership is not.
break_points	Optional estimated dates. Numeric values are **one-based row indices in y**, even when time is supplied. Alternatively supply Date or POSIXct values matching time exactly. Names, if supplied, must be Te, Tc, Tr; otherwise this order is used. Use NA to omit an estimate.
set_index	Optional indices mapping membership entries to rows of y: a common integer vector or a named list with the same names as conf_sets. Required for sets calculated on shorter windows. Without it each vector must have length(y) entries. Indices must be strictly increasing and unique.
time	Optional strictly increasing numeric, Date or POSIXct vector of length length(y). Defaults to time(y) for a ts object or observation numbers.
plot_type	Style of the resulting graph: paper or presentation.
date_first	Optional first date, used with date_by to construct a regular Date axis. Cannot be combined with time.
date_format	Format of calendar labels.
date_by	Time increment passed to seq.Date, such as "month".
colours	One colour or one colour per confidence set; names may be used.
conf_alpha	Opacity of the confidence bands, between zero and one.
shade_phases	Also show pale estimated phase spans between Te and Tc, between Tc and Tr, and after Tr. Requires all three break_points.
phase_alpha	Opacity of these optional phase spans.
y_label	Optional Y-axis label.
title	Optional plot title.

Details

These functions visualize supplied sets; they do not estimate dates or compute confidence levels. A disconnected set is never replaced by its convex hull. For bands, each accepted observation occupies a cell bounded by midpoints between adjacent observed times. The first and last cells extend by half of the adjacent time gap. Thus singleton sets remain visible. Connectivity is defined by consecutive rows of y, not by consecutive calendar days.

Bands describe uncertainty about a **date**, not the duration of a phase. Optional phase shading is determined by the point estimates and is labelled separately in the caption. The three supplied sets need not constitute a joint confidence region; this function does not infer simultaneous coverage.

Index mapping is explicit. When a bubble function receives y[a:b] including its initial lag observation, its length b-a membership vector is mapped with set_index = seq.int(a + 1L, b). For a full series including y0, estimates from segments.NBCN() must be mapped to displayed rows by adding one. Do not apply this offset to objects already indexed by rows of y.

Value

A ggplot object, displayed by R when evaluated at the console. Inside loops or functions use print(). The confsets_data attribute contains the time grid, membership table, connected components, mapped estimates and optional phase spans, for inspecting the exact plotted data.

References

Kurozumi, E. and Skrobotov, A. (2026). Confidence Sets for the Emergence, Collapse, and Recovery Dates of a Bubble. Econometric Reviews. doi:10.1080/07474938.2026.2700653.

See Also

plot_confsets(), plot_bubble(), cs.bubble.emerge()

Examples

```
t <- seq_len(100)
y <- sin(t / 8) + pmax(t - 30, 0) / 15 - pmax(t - 65, 0) / 8
sets <- list(Te = t %in% c(26:33, 38), Tc = t %in% 62:68,
             Tr = t %in% 83:91)
plot_bubble_confsets(y, sets, break_points = c(30, 65, 87))
```

---

plot_confsets *Plotting confidence sets in rows below a time series*

Description

Display a time series and aligned horizontal confidence sets, with one row per method or break date. Gaps and singleton sets are preserved.

Usage

```
plot_confsets(
  y,
  conf_sets,
  break_points = NULL,
  set_index = NULL,
  time = NULL,
  plot_type = c("paper", "presentation"),
  date_first = NULL,
  date_format = "%Y-%m",
  date_by = "month",
  colours = "#BD2A2A",
  relative_heights = c(2, 1),
  x_label = NULL,
  y_label = NULL,
  title = NULL,
  empty_label = "Empty set"
)
```

Arguments

y	Numeric time series or a one-column matrix. Missing observations are allowed and retain their original positions.
conf_sets	Named list of binary or logical membership vectors or one-column matrices. Names are used as row labels, in the supplied order.

break_points	Optional estimate per row, or one common estimate for all rows. Numeric values are row indices in y; Date/POSIXct values must match time. A named vector must match conf_sets; NA omits the corresponding marker.
set_index	Optional indices mapping membership entries to rows of y: a common integer vector or a named list with the same names as conf_sets. Required for sets calculated on shorter windows. Without it each vector must have length(y) entries. Indices must be strictly increasing and unique.
time	Optional strictly increasing numeric, Date or POSIXct vector of length length(y). Defaults to time(y) for a ts object or observation numbers.
plot_type	Style of the resulting graph: paper or presentation.
date_first	Optional first date, used with date_by to construct a regular Date axis. Cannot be combined with time.
date_format	Format of calendar labels.
date_by	Time increment passed to seq.Date, such as "month".
colours	One colour or one colour per row; names may be used.
relative_heights	Relative heights of the time-series and set panels.
x_label	Optional label for the shared time axis.
y_label	Optional Y-axis label.
title	Optional plot title.
empty_label	Label printed in a row with no accepted date.

Details

Computation of confidence sets is separate from plotting. Accepted consecutive observations are drawn as segments between their first and last times; a singleton is drawn as a point. Empty sets retain their labelled rows. Estimates, when supplied, are shown as black diamonds on the corresponding rows. Both panels have identical time limits. See plot_bubble_confsets() for the explicit mapping of membership vectors from shorter windows to rows of y.

Value

A patchwork object with two aligned ggplot panels. Use print() to display it in a loop or function, and ggplot2::ggsave() to save it. The confsets_data attribute contains the exact components and estimates plotted.

References

Eo, Y. and Morley, J. (2015). Likelihood-ratio-based confidence sets for the timing of structural breaks. Quantitative Economics, 6, 463-497. doi:10.3982/QE186. Kurozumi, E. and Skrobotov, A. (2018). Confidence Sets for the Break Date in Cointegrating Regressions. Oxford Bulletin of Economics and Statistics, 80, 514-535. doi:10.1111/obes.12223.

See Also

plot_bubble_confsets(), coint.conf.sets(), fcst.conf.set.EoMorley()

Examples

```
t <- seq_len(100)
y <- sin(t / 8)
sets <- list("Break 1" = t %in% 20:35,
             "Break 2" = t %in% c(60:68, 75:78, 85))
plot_confsets(y, sets, break_points = c(28, 64))
```

`PY.statistic` *Perron-Yabu (2009) statistic for break at unknown date.*

Description

Perron-Yabu (2009) statistic for break at unknown date.

Usage

```
PY.statistic(
  y,
  const = FALSE,
  trend = FALSE,
  criterion = "aic",
  trim = 0.15,
  max.lag = trunc(12 * (length(y)/100)^(1/4))
)
```

Arguments

`y`	A time series of interest.
`const`, `trend`	Allowing the break in constant or trend.
`criterion`	Needed information criterion: aic, bic, hq or lwz.
`trim`	A trimming parameter to determine the lower and upper bounds for a possible break point.
`max.lag`	The maximum possible lag in the model.

Value

A list of the estimated Wald statistic as well as its c.v.

References

Perron, Pierre, and Tomoyoshi Yabu. "Testing for Shifts in Trend With an Integrated or Stationary Noise Component." Journal of Business & Economic Statistics 27, no. 3 (July 2009): 369–96. https://doi.org/10.1198/jbes.2009.07268.

Skrobotov, Anton. "Survey on structural breaks and unit root tests." Applied Econometrics 58 (2020): 96–141. https://doi.org/10.22394/1993-7601-2020-58-96-141.

`uroot.ADF` *A simple implementation of ADF test*

Description

A function for ADF test with the ability to select the number of lags. Lags are selected by informational criterions which can be modified as in Ng and Perron (2001) and Cavaliere et al. (2015).

Usage

```
uroot.ADF(
  y,
  const = TRUE,
  trend = FALSE,
  max.lag = 0,
  criterion = NULL,
  modified.criterion = FALSE,
  rescale.criterion = FALSE,
  recursive = FALSE,
  cc = 0,
  gamma = 0,
  trim = 0.15,
  boot.p = FALSE,
  boot.iter = 999
)
```

Arguments

`y`	A time series of interest.
`const, trend`	Whether a constand and trend are to be included.
`max.lag`	Maximum lag number.
`criterion`	A criterion used to select number of lags. If lag selection is not needed keep this NULL.
`modified.criterion`	Whether the unit-root test modificaton is needed.
`rescale.criterion`	Whether the rescaling informational criterion is needed. Designed to cope with heteroscedasticity in residuals.
`recursive`	Whether a recursive detrending should be applied. See Smeekes (2013) for details.
`cc`	A filtration parameter used to construct an autocorrelation coefficient.
`gamma`	Detrending type selection parameter. If 0 the OLS detrending is applied, if 1 the GLS detrending is applied, otherwise the autocorrelation coefficient is calculated as $1 + c^{\gamma}T^{-\gamma}$.
`trim`	A trimming parameter.
`boot.p`	Whether bootstrapped p-values should be returned.
`boot.iter`	The number of bootstrap iterations.

Details

A function for ADF test with the ability to select the number of lags. Lags are selected by informational criterions which can be modified as in Ng and Perron (2001) and Cavaliere et al. (2015).

It's an ordinary Augmented Dickey-Fuller test using OLS estimation under the hood. Due to the Frisch-Waugh-Lovell theorem we first detrend y and then apply the test to the detrended series.

Value

A list containing:

- y,
- const,
- trend,
- residuals,
- coefficient estimates,
- t-statistic value,
- critical value,
- Number of lags,
- indicator of stationarity.

References

Cavaliere, Giuseppe, Peter C. B. Phillips, Stephan Smeekes, and A. M. Robert Taylor. "Lag Length Selection for Unit Root Tests in the Presence of Nonstationary Volatility." Econometric Reviews 34, no. 4 (April 21, 2015): 512–36. https://doi.org/10.1080/07474938.2013.808065.

Ng, Serena, and Pierre Perron. "Lag Length Selection and the Construction of Unit Root Tests with Good Size and Power." Econometrica 69, no. 6 (2001): 1519–54. https://doi.org/10.1111/1468-0262.00256.

Taylor, A. M. Robert. "Regression-Based Unit Root Tests With Recursive Mean Adjustment for Seasonal and Nonseasonal Time Series." Journal of Business & Economic Statistics 20, no. 2 (April 2002): 269–81. https://doi.org/10.1198/073500102317352001.

MacKinnon, James G. "Critical Values for Cointegration Tests." Working Paper. Economics Department, Queen's University, January 2010. https://ideas.repec.org/p/qed/wpaper/1227.html.

Smeekes, Stephan. "Detrending Bootstrap Unit Root Tests." Econometric Reviews 32, no. 8 (July 2013): 869–91. https://doi.org/10.1080/07474938.2012.690693.

Elliott, Graham, Thomas J. Rothenberg, and James H. Stock. "Efficient Tests for an Autoregressive Unit Root." Econometrica 64, no. 4 (1996): 813–36. https://doi.org/10.2307/2171846.

---

uroot.CHLT *MDF test for a single break and possible heteroscedasticity*

Description

MDF test for a single break and possible heteroscedasticity

Usage

```
uroot.CHLT(y, max.lag = NULL, trim = 0.15, boot.cv = FALSE, boot.iter = 999)
```

Arguments

y	A time series of interest.
max.lag	The maximum possible lag.
trim	Trimming parameter for lag selection
boot.cv	Whether we need bootstrapped critical values.
boot.iter	Number of bootstrap iterations.

Details

The code provided is the original GAUSS code by Cavaliere et al. ported to R.

Value

An object of type mdfCHLT. It's a list of four sublists each containing:

- The value of MZ_α, MSB, MZ_t, or ADF,
- The asymptotic c.v.,
- The bootstrapped c.v.

References

Cavaliere, Giuseppe, David I. Harvey, Stephen J. Leybourne, and A.M. Robert Taylor. "Testing for Unit Roots in the Presence of a Possible Break in Trend and Nonstationary Volatility." Econometric Theory 27, no. 5 (October 2011): 957–91. https://doi.org/10.1017/S0266466610000605.

---

uroot.HLT — *Test for the break in trend under uncertainty over the order of integration of the errors.*

Description

Test for the break in trend under uncertainty over the order of integration of the errors.

Usage

```
uroot.HLT(y, const = FALSE, trim = 0.15)
```

Arguments

y	A time series of interest.
const	Whether a constant is allowed to change (break in intersept is allowed).
trim	The trimming parameter to find the lower and upper bounds of possible break dates.

Value

The value of test statistic.

References

Harvey, David I., Stephen J. Leybourne, and A. M. Robert Taylor. "Simple, Robust, And Powerful Tests Of The Breaking Trend Hypothesis." Econometric Theory 25 , Issue 4 (2009): 995-1029. https://doi.org/10.1017/S0266466608090385

`uroot.robust` *A wrapping function around MDF.1br.*

Description

A wrapping function around MDF.1br.

Usage

```
uroot.robust(
  y,
  const = TRUE,
  trend = FALSE,
  season = FALSE,
  trim = 0.15,
  criterion = "bic"
)
```

Arguments

`y`	A time series of interest.
`const, trend`	Whether the constant term and trend should be included.
`season`	Whether the seasonal adjustment is needed.
`trim`	Trimming value for a possible break date bounds.

References

Skrobotov, Anton. "On Trend Breaks and Initial Condition in Unit Root Testing" Journal of Time Series Econometrics, 2018, Issue 1. https://doi.org/10.1515/jtse-2016-0014

`uroot.robust.mlt` *A wrapping function around breaktest.KP and MDF.mlt.*

Description

A wrapping function around breaktest.KP and MDF.mlt.

Usage

```
uroot.robust.mlt(y, const = TRUE, season = FALSE, breaks = 2, trim = 0.15)
```

Arguments

y	a time series of interest.
const	Whether the constant term is allowed to change (Structural changes in intercept, otherwise Structural changes only in slope).
season	Whether the seasonal adjustment is needed.
breaks	Number of breaks.
trim	Trimming value for a possible break date bounds.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

References

Skrobotov, Anton. "On Trend Breaks and Initial Condition in Unit Root Testing" Journal of Time Series Econometrics, 2018, Issue 1. https://doi.org/10.1515/jtse-2016-0014

---

uroot.SADF *Supremum ADF tests*

Description

Supremum ADF tests

Usage

```
uroot.SADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = TRUE,
  add.p.value = TRUE,
  boot.p = FALSE,
  boot.iter = 999
)

uroot.GSADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = TRUE,
  add.p.value = TRUE,
  boot.p = FALSE,
  boot.iter = 999
)

uroot.STADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = FALSE,
  omega.est = TRUE,
  truncated = TRUE,
```

```
  is.reindex = TRUE,
  ksi.input = "auto",
  hc = 1,
  pc = 1,
  add.p.value = TRUE
)

uroot.GSTADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = FALSE,
  omega.est = TRUE,
  truncated = TRUE,
  is.reindex = TRUE,
  ksi.input = "auto",
  hc = 1,
  pc = 1,
  add.p.value = TRUE
)

uroot.sb.GSADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = TRUE,
  alpha = 0.05,
  boot.iter = 999,
  urs = TRUE,
  seed = round(10^4 * sd(y))
)

uroot.w.SADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = TRUE,
  alpha = 0.05,
  iter = 4 * 200,
  urs = TRUE,
  seed = round(10^4 * sd(y))
)

uroot.w.GSADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = TRUE,
  alpha = 0.05,
  iter = 4 * 200,
  urs = TRUE,
  seed = round(10^4 * sd(y))
)
```

Arguments

y	A time series of interest.

trim	A trimming parameter to determine the lower and upper bounds for a possible break point.
const	Whether the constant needs to be included.
add.p.value	Whether the p-value is to be returned. This argument is needed to suppress the calculation of p-values during the precalculation of tables needed for the p-values estimating.
boot.iter	Number of bootstrapping iterations.
omega.est	Whether the variance of Nadaraya-Watson residuals should be used.
truncated	Whether the truncation of Nadaraya-Watson residuals is needed.
is.reindex	Whether the Cavaliere and Taylor (2008) time transformation is needed.
ksi.input	The value of the truncation parameter. Can be either auto or the explicit numerical value. In the former case the numeric value is estimated.
hc	The scaling parameter for Nadaraya-Watson bandwidth.
pc	The scaling parameter for the estimated truncation parameter value.
alpha	A significance level of interest.
urs	Use union of rejections strategy.
seed	A seed parameter for the random number generator.
iter	Number of iterations.

Details

Tests with time transformation are the modified versions of the ordinary SADF and GSADF tests using Nadaraya-Watson residuals and reindexing procedure by Cavaliere and Taylor (2008).

Refactored original code by Kurozumi et al.

This set of unit root tests are developed for testing for bubbles under time-varying non-stationary volatility. Because the limiting distribution of the seminal Phillips et al. (2011) test depends on the variance function and usually requires a bootstrap implementation under heteroskedasticity, Kurozumi et al. (2022) construct the test based on a deformation of the time domain. The proposed test is asymptotically pivotal under the null hypothesis and its limiting distribution coincides with that of the standard test under homoskedasticity, so that the test does not require computationally extensive methods for inference.

uroot.SADF is a test statistic equal to the minimum value of uroot.ADF for subsamples starting at $t = 1$.

GSADF.test is a generalized version of SADF.test. Subsamples are allowed to start at any point between 1 and $T(1 - trim)$.

Value

uroot.GSADF returns an object of class bt_SADF and subclass bt_GSADF.

uroot.STADF returns an object of type bt_SADF and subclass bt_STADF. It's a list of:

y	
N	Number of observations,
trim	
const	
omega.est	

`truncated`

`is.reindex`

`new.index` the vector of new indices,

`ksi.input`

`hc`

`h.est`

`u.hat`

`pc`

`w.sq`

`t.values` vector of t-values,

`*.value` the value of the corresponding test statistic,

`u.hat.truncated`
truncated residuals if truncation was asked for,

`ksi\verb{, }sigma`
estimated values of the truncation parameter and resulting s.e. if `ksi.input` equals `auto`,

`eta.hat` the values of reindexing function if reindexing was asked for,

`\eqn{p}-value` if it was asked for.

uroot.GSTADF returns an object of type `bt_SADF` and subclass `bt_GSTADF`.

uroot.sb.GSADF returns an object of class `bt_SADF` and subclass `bt_sbGSADF`. It's a list of:

- `y`,
- main parameters, such as `trim`, `const`, `alpha`, `iter`,
- `seed`, supremum SBADF-statistic values incl. bootstrapped ones,
- p-value,
- indicator of explosive process.

If `urs=TRUE` then some additional values included:

- t-values,
- GSADF bootstrapped statistics,
- bootstrapped U-values.

uroot.w.SADF returns an object of class `bt_SADF` and subclass `bt_wSADF`. It's a list of:

`y`

`trim`

`const`

`alpha`

`iter`

`urs`

`seed`

`sigma.sq` the estimated variance,

`BZ.values` a series of BZ-statistic,

`supBZ.value` the maximum of `supBZ.values`,

`supBZ.bootstsrap.values`
bootstrapped supremum BZ values,

`supBZ.cr.value` supremum BZ α critical value,

`p.value`

,

`is.explosive` 1 if `supBZ.value` is greater than `supBZ.cr.value`.

if `urs` is `TRUE` the following items are also included:

`t.values` vector of t-values,

`SADF.value` the value of the SADF test statistic,

`SADF.bootstrap.values`
bootstrapped SADF values,

`SADF.cr.value` critical value of `SADF.value`.

`U.value` union test statistic value,

`U.bootstrap.values`
bootstrapped series of `U.value`,

`U.cr.value` critical value of `U.value`.

uroot.w.GSADF returns an object of type `bt_SADF` and subclass `bt_wGSADF`.

uroot.SADF returns an object of type `bt_SADF`. It's a list of:

- `y`,
- `trim`,
- `const`,
- vector of t-values,
- the value of the corresponding test statistic,
- p-value if it was asked for.

References

Cavaliere, Giuseppe, and A. M. Robert Taylor. "Time-Transformed Unit Root Tests for Models with Non-Stationary Volatility." Journal of Time Series Analysis 29, no. 2 (March 2008): 300–330. https://doi.org/10.1111/j.1467-9892.2007.00557.x.

Harvey, David I., Stephen J. Leybourne, and Yang Zu. "Sign-Based Unit Root Tests for Explosive Financial Bubbles in the Presence of Deterministically Time-Varying Volatility." Econometric Theory 36, no. 1 (February 2020): 122–69. https://doi.org/10.1017/S0266466619000057.

Kurozumi, Eiji, Anton Skrobotov, and Alexey Tsarev. "Time-Transformed Test for Bubbles under Non-Stationary Volatility." Journal of Financial Econometrics, April 23, 2022. https://doi.org/10.1093/jjfinec/nbac004.

Index

AIC.bt_ols *(info.criterions)*, 43

BIC.bt_ols *(info.criterions)*, 43
bootstrap, 2
breaktest.KP, 3, *55*
bubbles.nbcn *(cs.bubble.emerge)*, 13

coint.conf.sets, 4
coint.conf.sets(), *50*
coint.conf.sets.KY, 6
coint.CSS, 7
coint.GH, 10
coint.PR, 11
coint.VECM, 12
cs.bubble.collapse *(cs.bubble.emerge)*, 13
cs.bubble.emerge, 13
cs.bubble.emerge(), *49*
cs.bubble.recovery *(cs.bubble.emerge)*, 13
cset_break_coint *(coint.conf.sets)*, 4

fcst.AveLR, 15
fcst.AveW, 16
fcst.BP.simulate, 17, *36*
fcst.BP.tables, *17*
fcst.CIavg, 18, *29*
fcst.compare, 20, *25, 26*, *31*, *39*
fcst.conf.int.Bai, *19*, 22
fcst.conf.set.EoMorley, *19*, 23, *23*
fcst.conf.set.EoMorley(), *50*
fcst.DGP, 25
fcst.ExpS, 27, *31*
fcst.FGLS, 28
fcst.forecast, *20, 21*, *27, 28*, 30, *33, 34*, *39*, *41*
fcst.Pooled *(fcst.WA)*, 38
fcst.select, *31*, 32
fcst.simulate *(fcst.DGP)*, 25
fcst.stein, 33
fcst.test.BP, 35
fcst.test.DM, *21*
fcst.test.Wald, *28*, 37, *39*
fcst.WA, 38
fcst.WGLS.cv, 40
fcst.window, *31*
fcst.window.cv, *27*
fcst.window.LS, *22*, *34*, *40*

get.cv.KY, 41
get.cv.Wald *(get.pv.Wald)*, 42
get.pv.Wald, 42

HAC.variance, *22*, *24*, *33*
HQIC.bt_ols *(info.criterions)*, 43

info.criterions, 43

KP.seq.statistic, 44

LWZ.bt_ols *(info.criterions)*, 43

MDF.1br, 45, *55*
MDF.mlt, 46, *55*

OLS.reg, *24, 25*, *27, 28*, *37*

plot_bubble, 46
plot_bubble(), *49*
plot_bubble_confsets, 47
plot_bubble_confsets(), *50*
plot_confsets, 49
plot_confsets(), *49*
PY.statistic, 51

segments.AR1, *14*
segments.AR1 *(cs.bubble.emerge)*, 13
segments.NBCN, *14*
segments.NBCN *(cs.bubble.emerge)*, 13
SSR.recursive, *24*

uroot.ADF, 52, *58*
uroot.CHLT, 53
uroot.GSADF, *58*
uroot.GSADF *(uroot.SADF)*, 56
uroot.GSTADF, *59*
uroot.GSTADF *(uroot.SADF)*, 56
uroot.HLT, 54
uroot.robust, 55

uroot.robust.mlt, 55
uroot.SADF, 56, *60*
uroot.sb.GSADF, *59*
uroot.sb.GSADF *(*uroot.SADF*)*, 56
uroot.STADF, *58*
uroot.STADF *(*uroot.SADF*)*, 56
uroot.w.GSADF, *60*
uroot.w.GSADF *(*uroot.SADF*)*, 56
uroot.w.SADF, *59*
uroot.w.SADF *(*uroot.SADF*)*, 56

WLS.reg, *27*, *41*